

嵌入式系统

第四讲 ARM硬件结构（一）

教师：郭玉臣

Mail:yuchenguo@tongji.edu.cn



LPC2000系列ARM硬件结构

- **1.LPC2000系列简介**
- **2.引脚描述**
- **3.存储器寻址**
- **4.系统控制模块**
- **5.存储器加速模块 (MAM)**
- **6.外部存储器控制器 (EMC)**
- **7.引脚连接模块**
- **8.GPIO**
- **9.向量中断控制器**
- **10.外部中断输入**
- **11.定时器0和定时器1**
- **12.SPI接口**
- **13.I²C接口**
- **14.UART(0、1)**
- **15.A/D转换器**
- **16.看门狗**
- **17.脉宽调制器(PWM)**
- **18.实时时钟**



4.1 LPC2000系列简介

- 简介

LPC2000系列微控制器基于ARM7TDMI-S CPU内核。支持ARM和Thumb指令集，芯片内集成丰富外设，而且具有非常低的功率消耗。使该系列微控制器特别适用于工业控制、医疗系统、访问控制和POS机等场合。

LPC2000 系列 ARM 产品

- **LPC2100 系列**
- **LPC2200 系列**
- **LPC2300 系列**
- **LPC2400 系列**
- **LPC2800 系列**

4.1 LPC2100/2200系列简介

- 器件信息

器件	引脚数	片内RAM	片内Flash	10位AD通道数	备注
LPC2114	64	16KB	128KB	4	—
LPC2124	64	16KB	256KB	4	—
LPC2210	144	16KB	—	8	带外部 存储器接口
LPC2220	144	64KB	—	8	
LPC2212	144	16KB	128KB	8	
LPC2214	144	16KB	256KB	8	

关于LPC2000其它器件的介绍请登录www.zlgmcu.com “LPC2000系列ARM”专栏

芯片内部结构

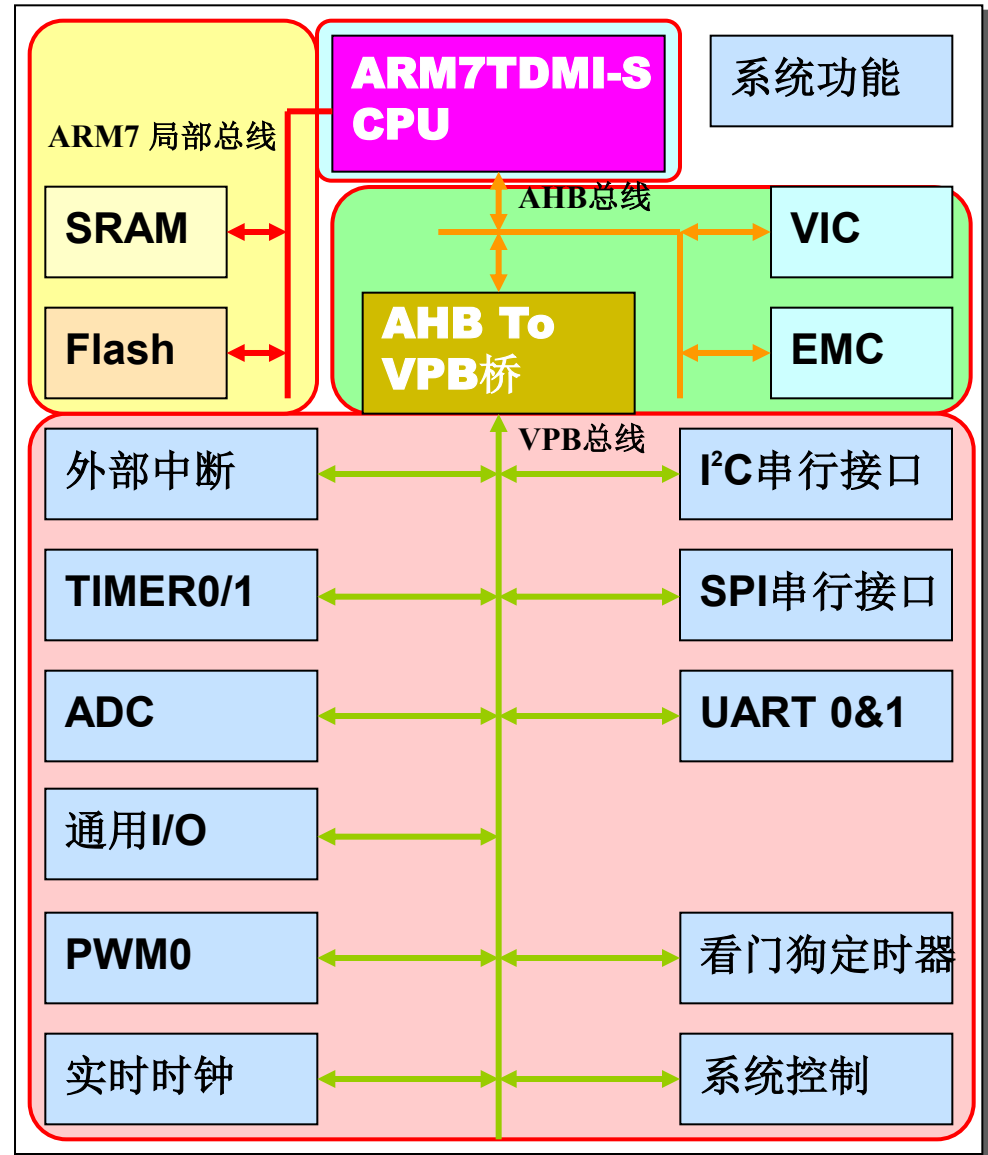
LPC2000系列微控制器包含4大部分:

①支持仿真的ARM7TDMI-S CPU

②与片内存储器控制器接口的ARM7局部总线

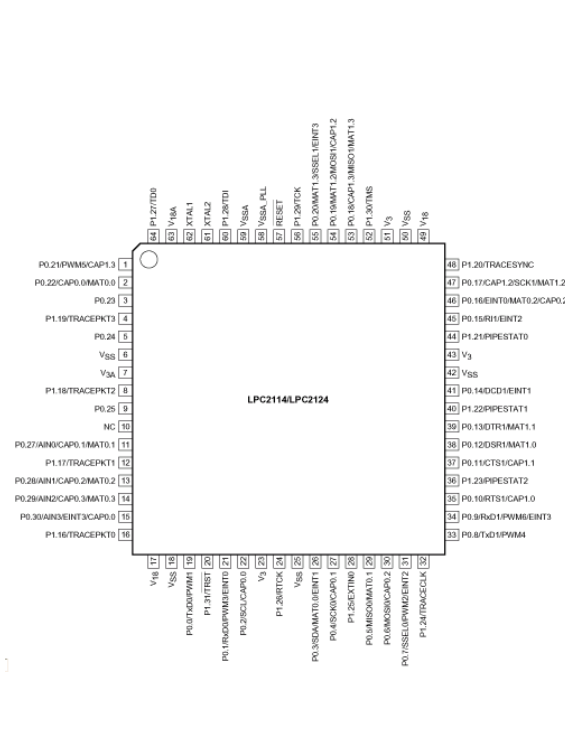
③与中断控制器接口的AMB A高性能总线(AHB)

④连接片内外设功能的VLSI 外设总线(VPB)

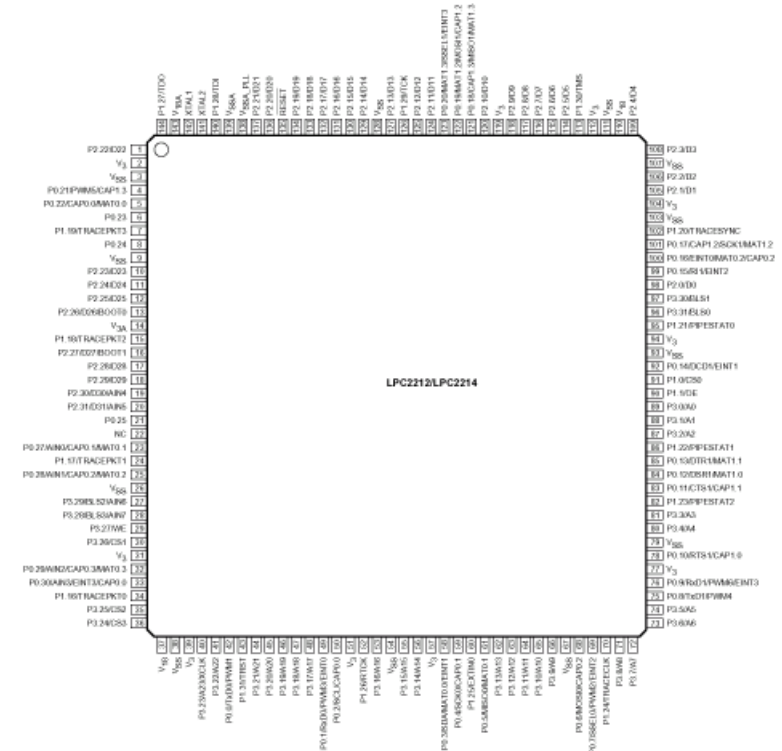


4.2 引脚描述

• LPC2000系列芯片外形

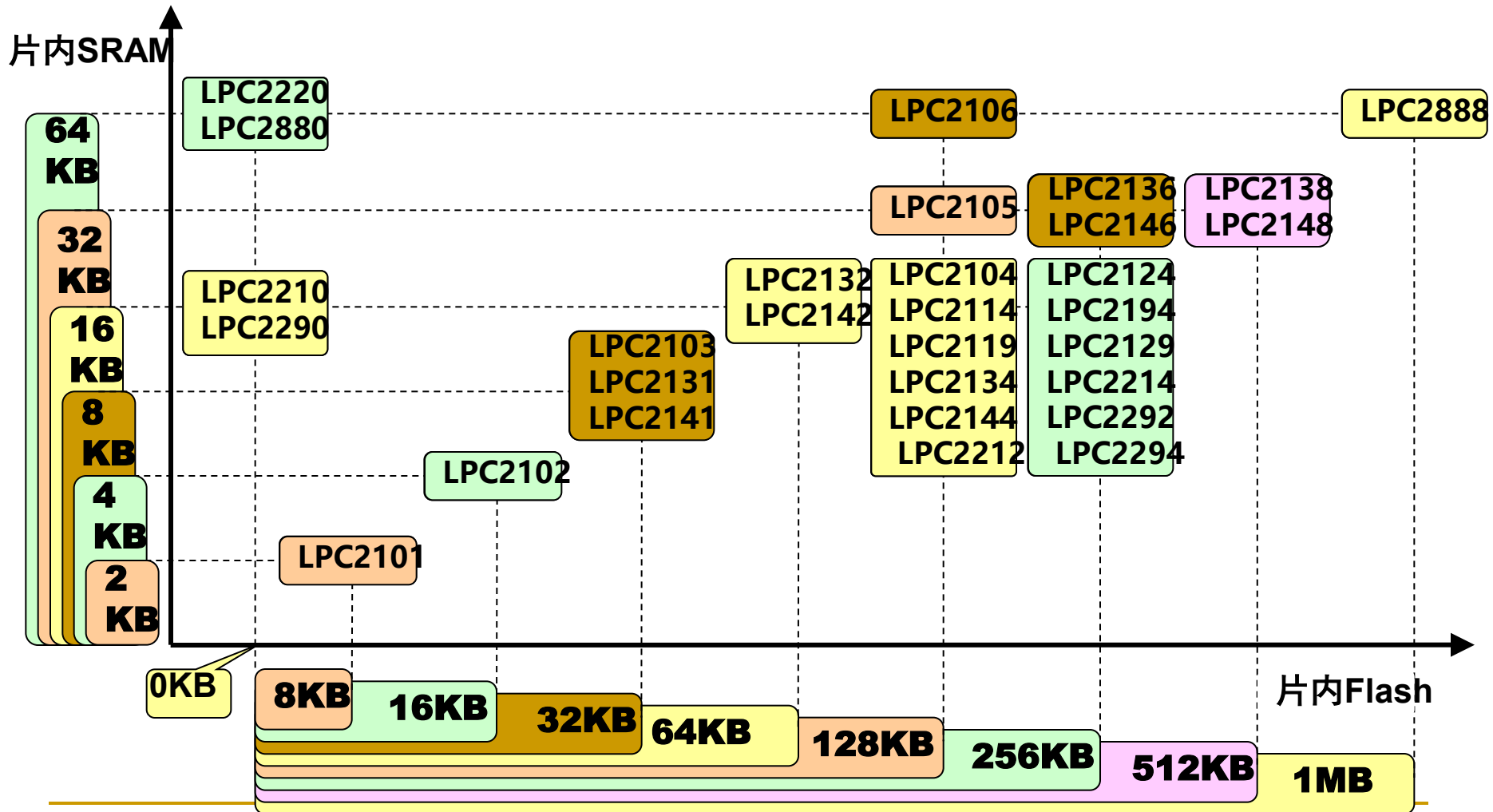


LPC2114/2124



LPC2210/2220/2212/2214

LPC2000系列微处理器的片内存储器大小



4.3 存储器



4.3 存储器寻址

- 0. 存储器概述
 - 1. 片内存储器
 - 2. 片外存储器
 - 3. 存储器映射
 - 4. 预取指中止和数据中止异常
 - 5. 存储器重映射及引导块
 - 6. 启动代码相关部分
-



4.3.0 存储系统概述

- 1 半导体存储器
- 2 存储器的性能指标
- 3 常用的几种存储器

1 半导体存储器

- 作用
 - 存放程序与数据
- 嵌入式系统存储器的特殊要求
 - 集成度高、体积小、功耗低
- 发展趋势
 - 片上集成
 - Why? ——性能、可靠性、成本
- 片内存储器 VS 片外存储器
 - 片内：速度快、容量小
 - 片外：容量大、速度慢



2 存储器的性能指标

- 性能指标
 - 只读性
 - 挥发性
 - 存储容量
 - 速度
 - 功耗
 - 可靠性

3 常用的几种存储器

■ SRAM（静态随机存储器）

- 存储密度小

 - 6管结构，占用较大芯片面积

- 价格较高

- 功耗较高

- 容量较小

- 存取速度快

- 接口时序简单

3 常用的几种存储器（续）

■ DRAM（动态随机存储器）

- 存储密度大
 - 单管结构
- 单位存储成本较低
- 功耗较低
- 容量较大
- 接口时序复杂
 - 需要刷新电路



3 常用的几种存储器（续）

■ EEPROM

- ❑ 非挥发
- ❑ 存储密度小
- ❑ 单位存储成本较高
- ❑ 容量小
- ❑ 写入有限制，页写要等待
- ❑ 接口时序简单，一般采用串行接口
- ❑ 小量参数存储

3 常用的几种存储器（续）

■ Flash（闪存存储器）

- ❑ 非挥发
- ❑ 存储密度大
- ❑ 单位存储成本较低
- ❑ 容量大
- ❑ 接口时序复杂——需要擦除及Block写
- ❑ NOR Flash & NAND Flash

3 常用的几种存储器（续）

■ FRAM（铁电存储器）

- 非挥发
- 功耗低
- 读写速度快
- 接口时序简单
 - 类似SRAM接口
- 成本高

3 常用的几种存储器（续）

■ 并行接口存储器

□ 引脚数目多——三大总线

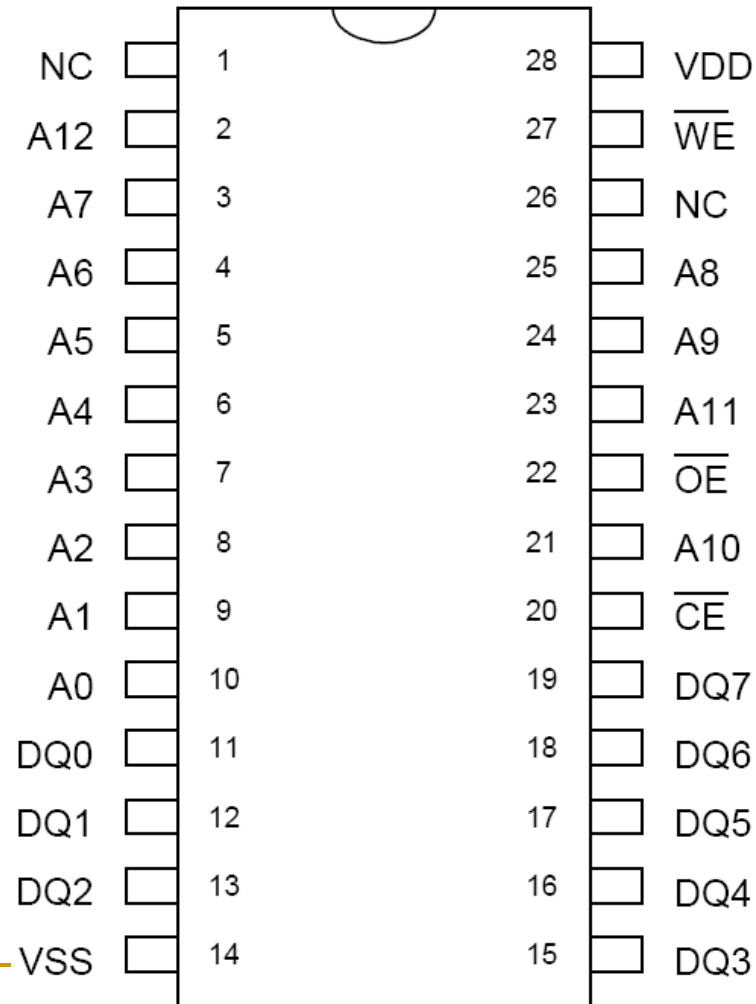
- 地址、控制总线
- 数据总线（8/16/32位）

□ 存储容量大

- 适用于大容量存储场合

□ 存取速度快

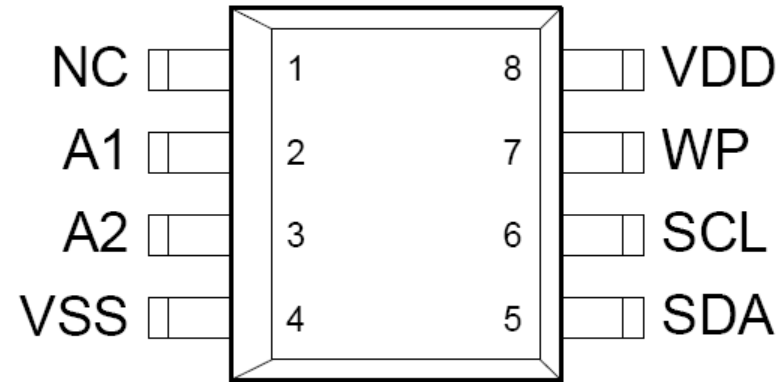
□ 接口时序复杂，编程透明



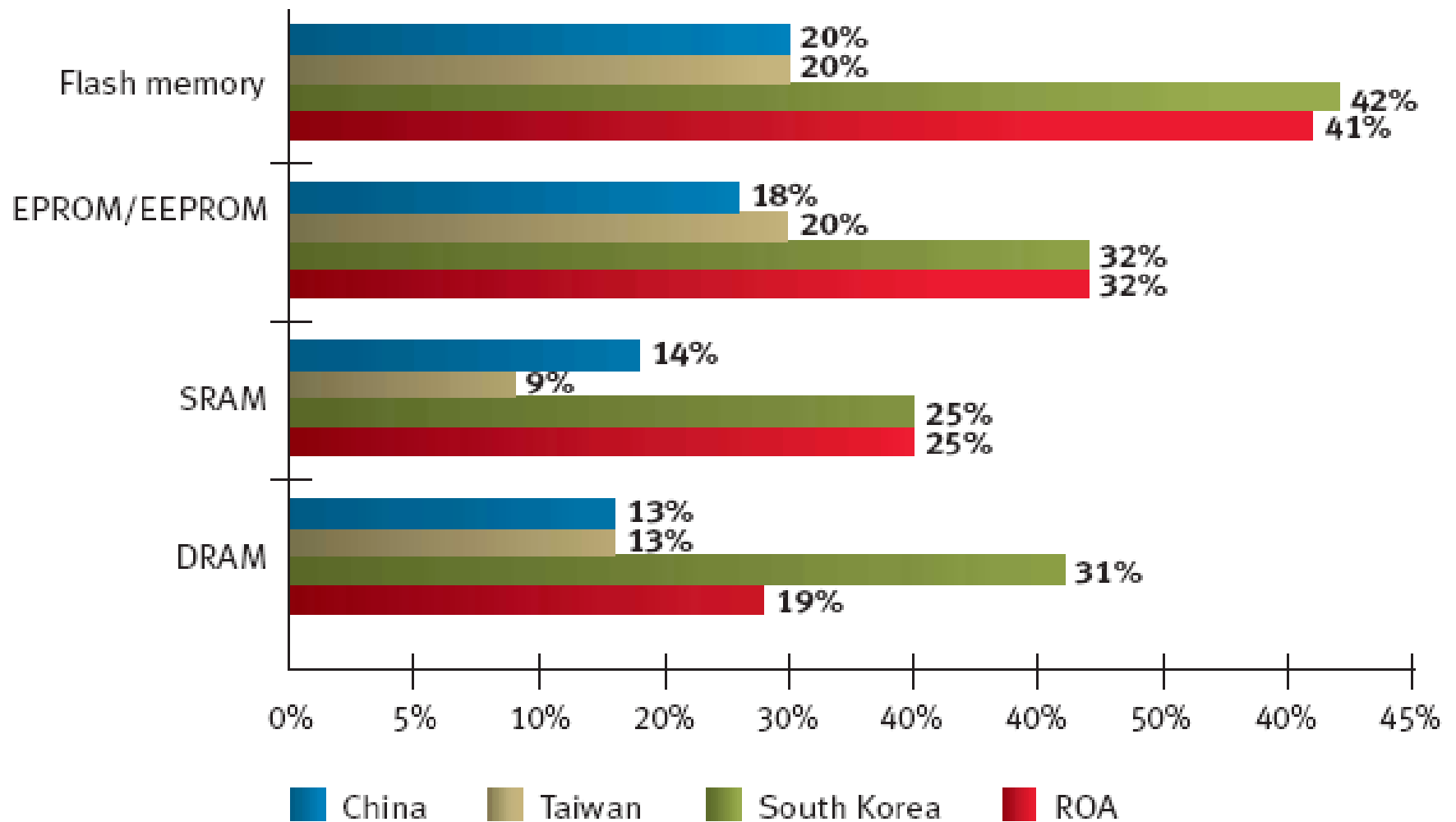
3 常用的几种存储器（续）

■ 串行接口存储器

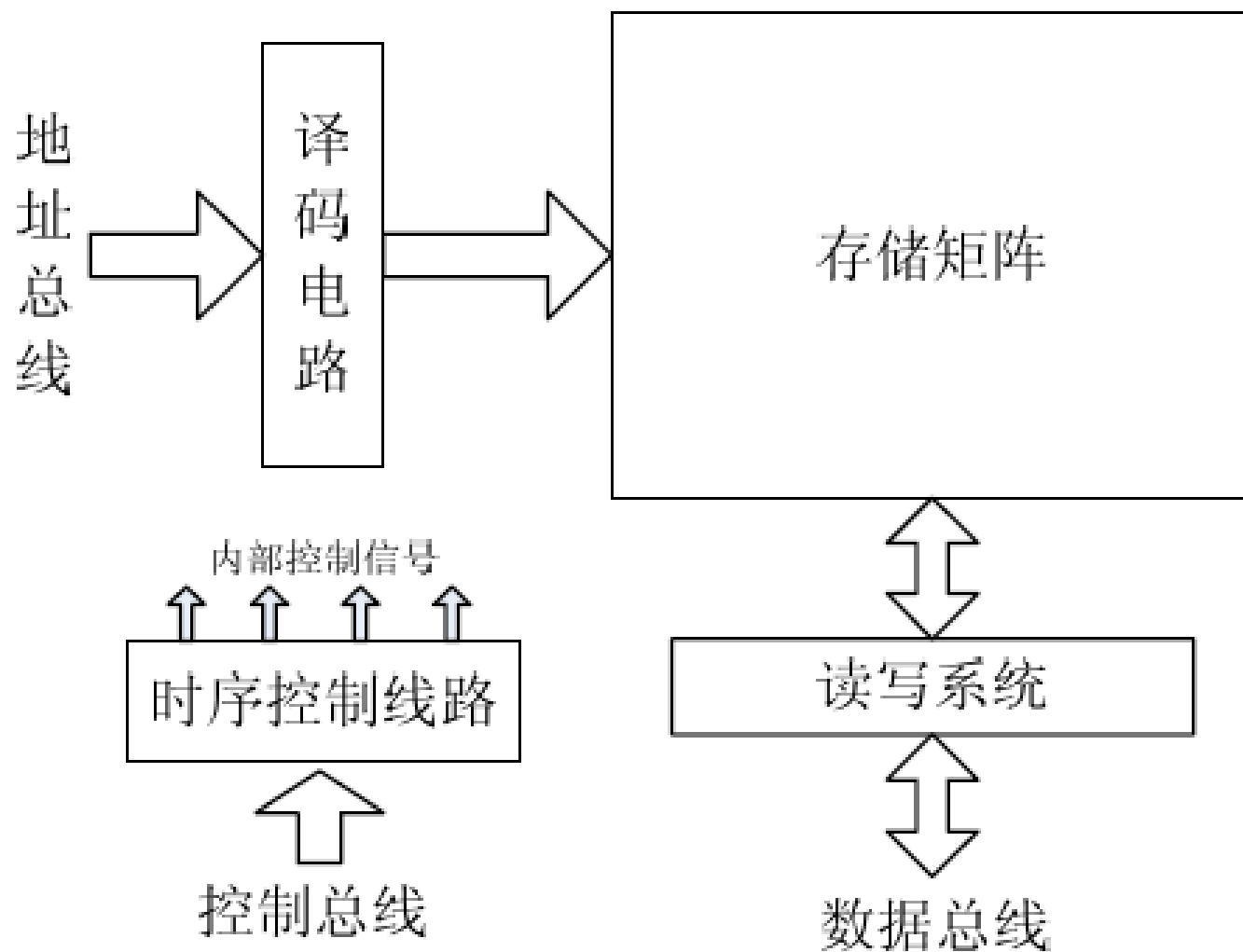
- 引脚数目 **极少**
- 存储容量 **较小**
 - 适用于较小存储容量场合
- 存取速度 **较慢**
- 使用 **串行接口通信**，**接口标准化**，**编程不透明**
 - I2C、SPI
 - 可以用软件通过GPIO模拟



2004年亚洲存储器选用情况



4存储器的结构



5 嵌入式系统存储器子系统

- 与通用计算机并无本质区别，但有自身特点
 - 存储密度要求
 - 功耗要求
 - 片内集成存储器——彻底抛弃片外存储器
 - 一般焊接在板子上，较少采用内存条
- 存储空间分配
 - 嵌入式系统一般具有多种类型存储器
 - 支持多种存储器扩展
 - 接口灵活、可配置



4.3 存储器寻址

- 1.片内存储器
- 2.片外存储器
- 3.存储器映射
- 4.预取指中止和数据中止异常
- 5.存储器重映射及引导块
- 6.启动代码相关部分

4.3.1 片内存储器

- 片内FLASH程序存储器

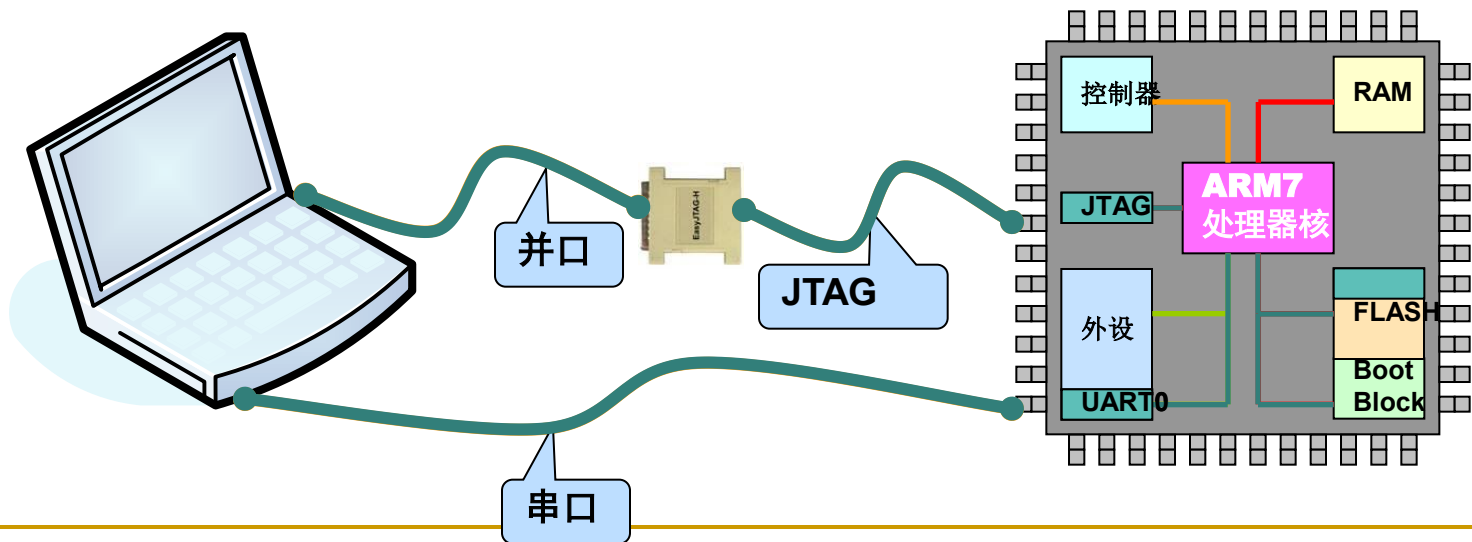
LPC2000系列中除了LPC2210/2220/2290外，其它的ARM微处理器内部都带有容量不等的Flash，这为ARM芯片的单片应用带来可能。

片内Flash通过128位宽度的总线与ARM内核相连，具有很高的速度，加上特有的存储器加速功能，因此可以将程序直接放在Flash上运行。

4.3.1 片内存储器

■ 片内Flash编程方法

3. 使用在线编程技术(即通过串口的JTAG接口下载程序Flash进行擦除和/或编程操作, 实现数据的存储和固件的现场升级。



4.3.1 片内存储器

■ 片内Flash编程方法

1. 使用JTAG仿真/调试器，通过芯片的JTAG接口下载程序；
2. 使用在系统编程技术（即ISP），通过UART0接口下载程序；
3. 使用在应用编程技术（即IAP），在用户程序运行时对Flash进行擦除和/或编程操作，实现数据的存储和固件的现场升级。



4.3.1 片内存储器

- 片内静态RAM

LPC2000系列微控制器的片内RAM为静态RAM (SRAM)，它们可用作代码和/或数据的存储。

SRAM支持8位、16位和32位的读写访问。



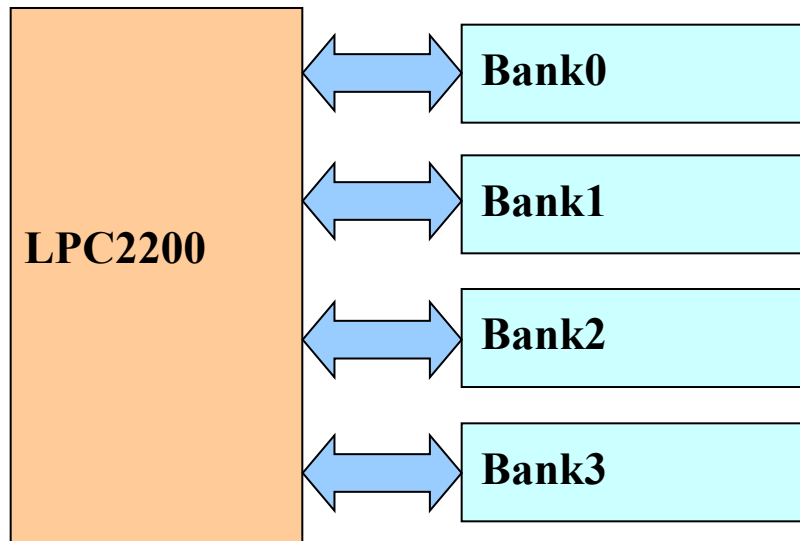
4.3 存储器寻址

- 1.片内存储器
 - 2.片外存储器
 - 3.存储器映射
 - 4.预取指中止和数据中止异常
 - 5.存储器重映射及引导块
 - 6.启动代码相关部分
-

4.3.2 片外存储器

■ 概述

在CPU外部扩展连接的存储器芯片称为片外存储器，这些器件通常都具有数据线、地址线和控制线等。主要器件有ROM、FLASH、SRAM等。



每个Bank

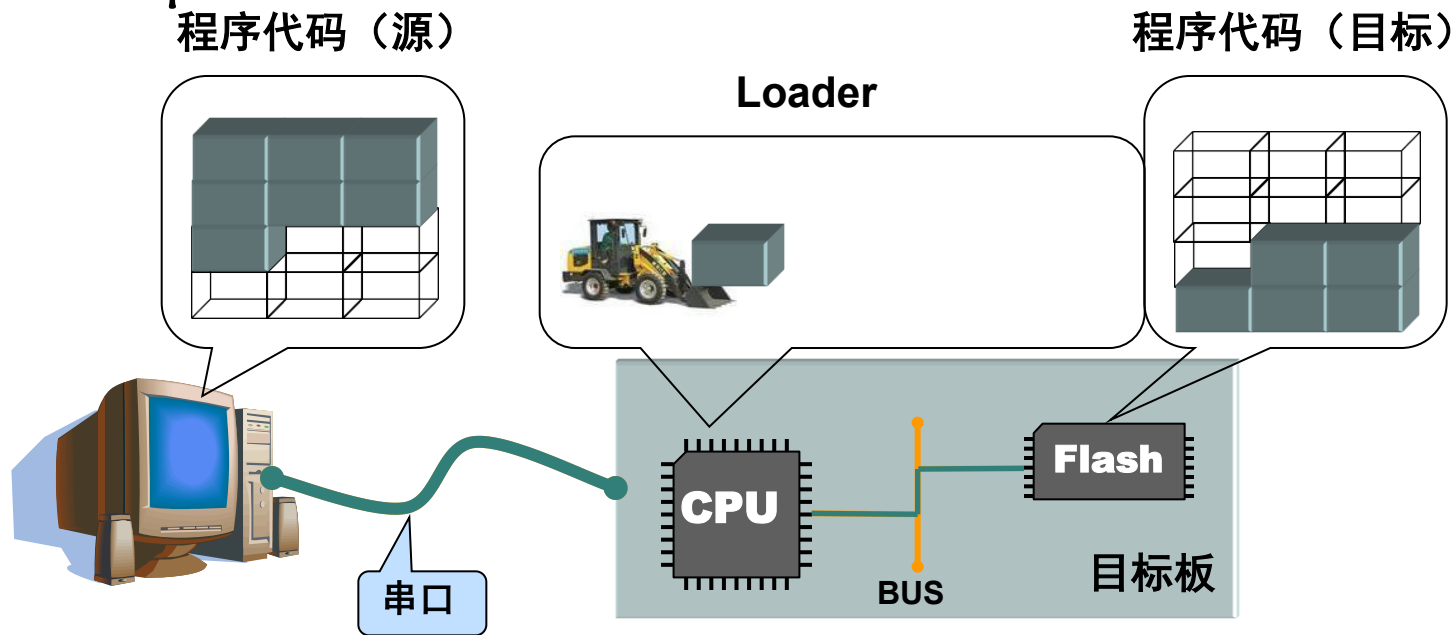
寻址空间：16M字节；

数据宽度：8/16/32位。

4.3.2 片外存储器

■ 片外Flash编程方法

Flash编程操作需要能在CPU内容Flash编程程序，通过它把这段代码（称为**装载程序**），接收的代码写入Flash中。





4.3 存储器寻址

- 1.片内存储器
 - 2.片外存储器
 - 3.存储器映射
 - 4.预取指中止和数据中止异常
 - 5.存储器重映射及引导块
 - 6.启动代码相关部分
-

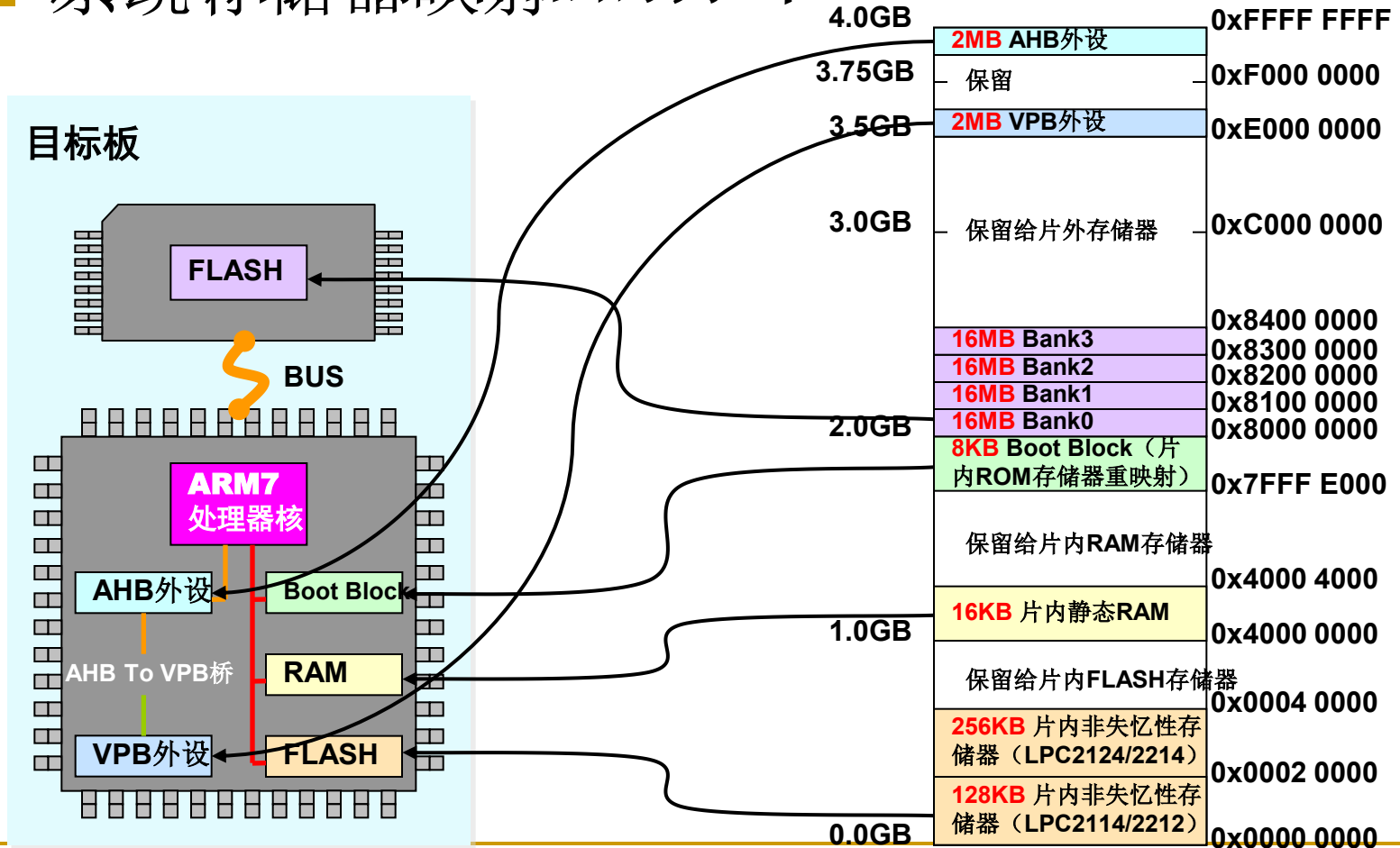
4.3.3 存储器映射

■ 概述

ARM芯片可以存在片内和片外存储器，这些存储器本身不具有地址信息，它们在芯片中的地址是由芯片厂家或用户分配的，那么**给物理存储器分配逻辑地址的过程称为存储器映射**。通过这些逻辑地址就可以访问到相应存储器的物理存储单元。

4.3.3 存储器映射

■ 系统存储器映射



4.3.3 存储器映射

■ AHB和VPB

AHB（先进的高性能总线）和VPB（VLSI外设总线）外设区域都为2M字节，可各分配128个外设。每个外设空间的规格都为16K字节，这样就简化了每个外设的地址译码。

注意： 外设寄存器的地址都是字对齐。AHB和VPB外设区域中不管是字还是半字，都是一次性访问。例如不可能对一个字寄存器的最高字节执行单独的读或写操作。

4.3.3 存储器映射

■ 外设存储器映射

4.0GB	2MB AHB外设	0xFFFF FFFF
3.75GB	保留	0xF000 0000
4.0GB		0xFFFF FFFF
3.5GB	AHB外设	0xE000 0000
4.0GB-2MB		0xFFE0 0000
3.0GB	保留给片外存储器	0xC000 0000
	保留	0x8400 0000
	16MB Bank3	0x8300 0000
	16MB Bank2	0x8200 0000
	16MB Bank1	0x8100 0000
	16MB Bank0	0x8000 0000
3.25GB	8KB Boot Block (片内ROM存储器重映射)	0x7FFF E000
	保留给片内RAM存储器	
	保留	0x4000 4000
1.0GB	16KB 片内静态RAM	0x4000 0000
3.5GB+2MB	保留给片内FLASH存储器	0xE020 0000
		0x0004 0000
3.5GB	VPB外设	0xE000 0000
		0x0002 0000
0.0GB	128KB 片内非失忆性存储器 (LPC2114/2212)	0x0000 0000

注：AHB和VPB均为128x16kB(2MB)范围。

4.3.3 存储器映射

■ AHB外设映射

4.0GB		0xFFFFF000
4.0GB-2MB	向量中断控制器	0xFFE0 0000 0xFFFFF000
	外部总线控制器	0xFFFFC000
3.75GB	(AHB外设#126) 未使用	0xFF6F 8000
	~(AHB外设 #1- AHB外设 #125)~ 未使用	
	(AHB外设#0) 未使用	0xFFE04000 0xE020 0000
3.5GB+2MB		
3.5GB		0xEBE0 0000

注：只有
LPC2200系列微处理
器有外部总线控制器

4.3.3 存储器映射

■ VPB外设映射

	系统控制模块 (VPB外设#127)	0xE01F FFFF
		0xE01F C000
4.0GB	 未使用	0xFFFF FFFF
		0xE006 C000
4.0GB-2MB	SSP (VPB外设#26)	0xFFE0 0000
		0xE006 8000
	 未使用	0xE006 0000
	I ² C1 (VPB外设#23)	0xE005 C000
3.75GB	 未使用	0xF000 0000
		0xE003 8000
	10位A/D (VPB外设#13)	0xE003 4000
		0xE000 8000
3.5GB+2MB	TIMER0 (VPB外设#1)	0xE020 0000
		0xE000 4000
3.5GB	看门狗定时器 (VPB外设#0)	0xE000 0000
		0xE000 0000



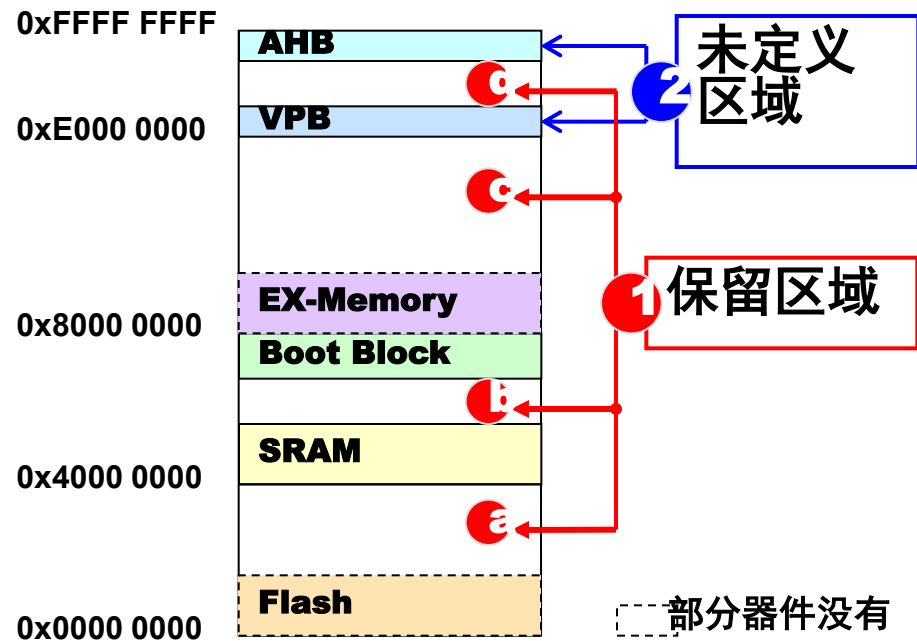
4.3 存储器寻址

- 1.片内存储器
 - 2.片外存储器
 - 3.存储器映射
 - 4.预取指中止和数据中止异常
 - 5.存储器重映射及引导块
 - 6.启动代码相关部分
-

4.3.4 预取指中止和数据中止异常

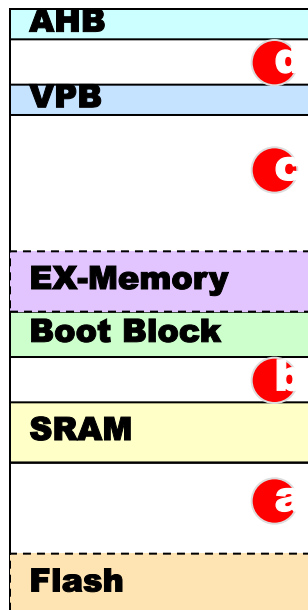
■ 概述

如果试图访问一个**保留区域地址**或**未分配区域地址**，ARM处理器将产生预取指中止或数据中止异常。



4.3.4 预取指中止和数据中止异常

■ 保留地址区域



● 片内非易失性存储器与片内SRAM之间保留给片内存储器的地址空间。

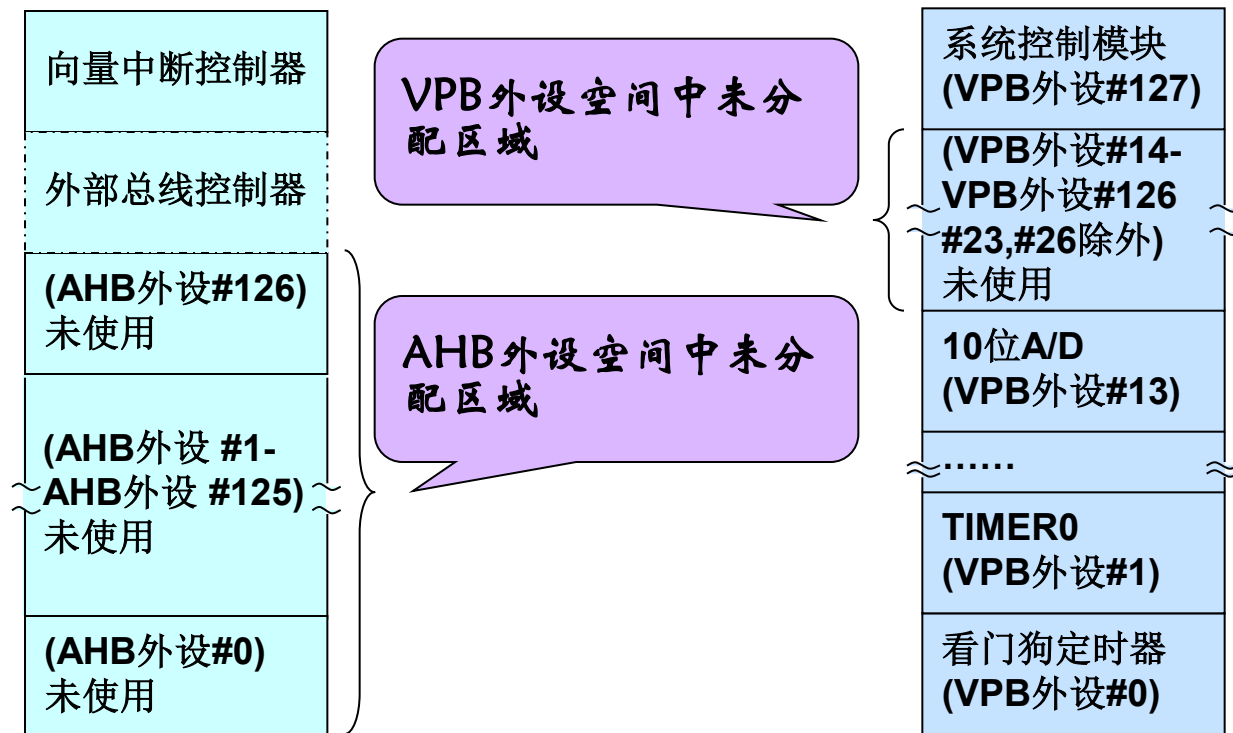
● 片内静态RAM与外部存储器之间保留给片内存储器的地址空间。

● 外部存储器区域中无法通过外部存储器控制器 (EMC) 来访问的地址空间。

● AHB和VPB空间的保留区域。

4.3.4 预取指中止和数据中止异常

■ 未分配地址区域





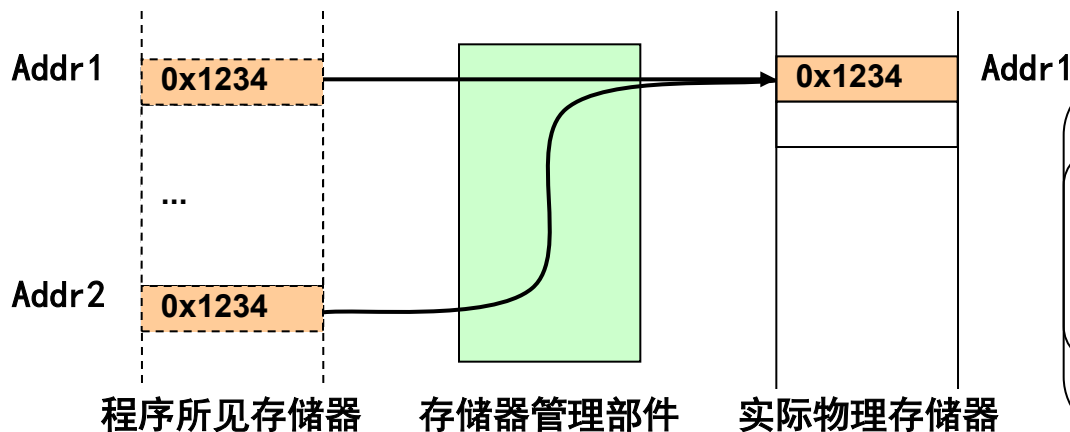
4.3 存储器寻址

- 1.片内存储器
 - 2.片外存储器
 - 3.存储器映射
 - 4.预取指中止和数据中止异常
 - 5.存储器重映射及引导块
 - 6.启动代码相关部分
-

4.3.5 存储器重映射及引导块

■ 存储器重映射

注意 经过存储器重映射后再映射的集称为**容进**
行重复制，它使得将物理存储单元出现多个存储单元，
 这种效果是使存储单元全部包含存储管理即“Block”
 的用于保存异常向量的少量存储单元。



实际物理存储单元通过
 存储器管理部件进行存储器
 重映射，获得逻辑地址
 Addr2。此时，该逻辑地址
 Addr1和Addr2可以访问同一
 实际物理存储单元。



4.3.5 存储器重映射及引导块

■ 引导块及其重映射

引导块 (Boot Block) 是芯片设计厂家在LPC2000系列ARM内部固化的一段代码，用户无法修改或删除。这段代码在芯片复位后被首先运行，其功能主要是：

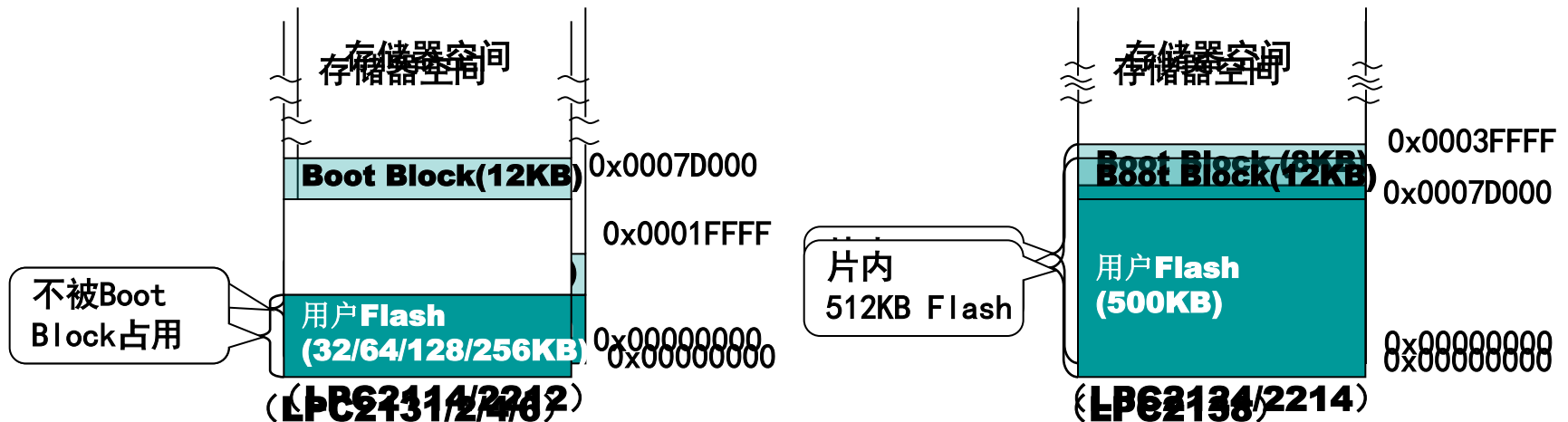
- 判断运行哪个存储器上的程序；
- 检查用户代码是否有效；
- 判断芯片是否被加密；
- 芯片的在应用编程(IAP)以及在系统编程功能(ISP)。

注意：部分器件内部虽然没有用户Flash空间（比如LPC2210/2220/2290），但它们仍然存在Boot Block，并且复位后会被首先运行。

4.3.5 存储器重映射及引导块

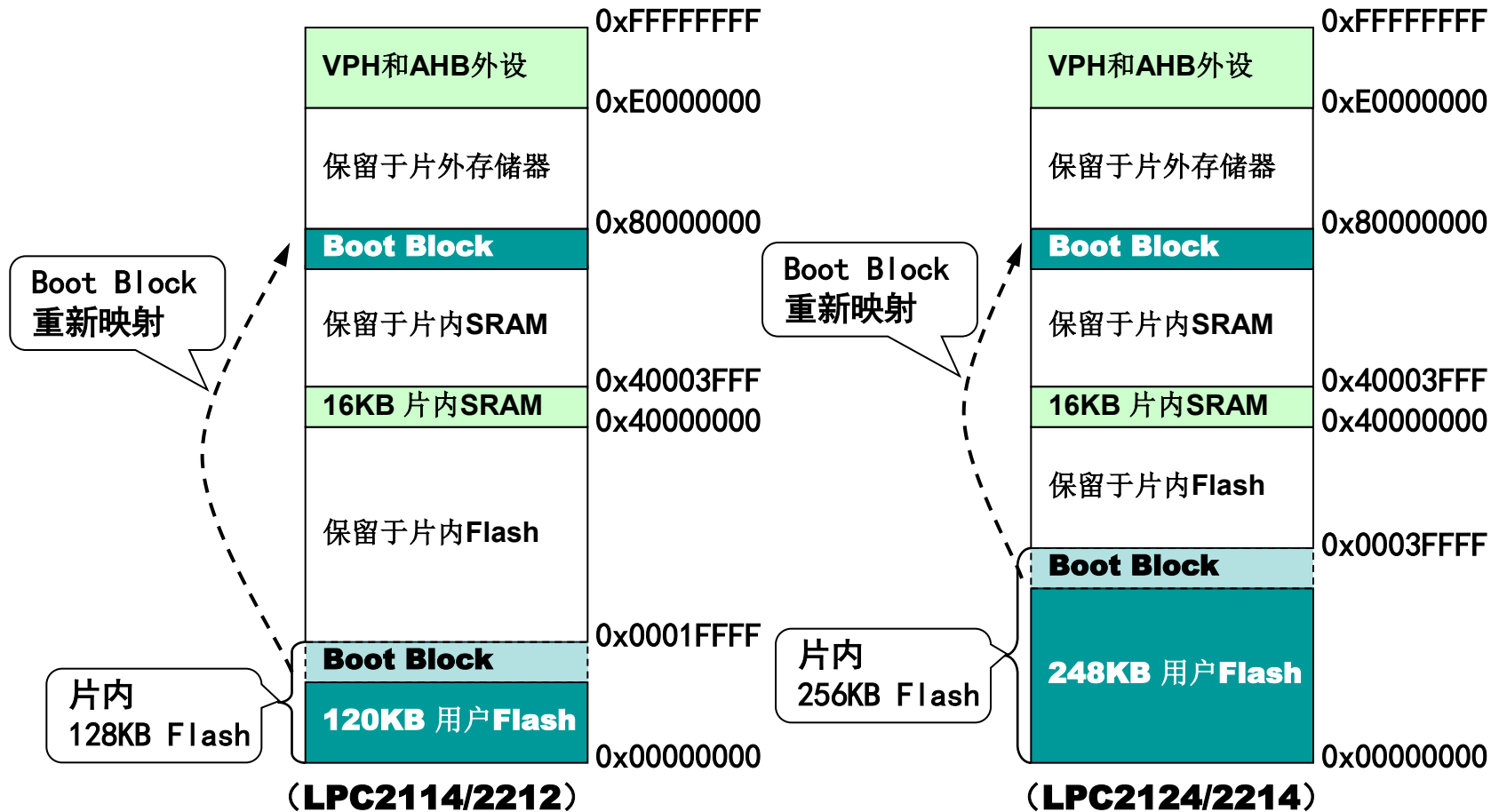
■ 引导块在存储器中的状态

LPC2130系列芯片的Boot Block为12KB大小，除了LPC2130与用户Flash空间外，该系列中其它的芯片不占用用户Flash空间。



4.3.5 存储器重映射及引导块

引导块（Boot Block）的重映射



4.3.5 存储器重映射及引导块

• 异常向量表概述

异常向量表位于存储器映射的0x0000~0x001C地址空间，定义了8个异常向量，每个异常向量占一个字。通常在每个**异常入口**放置一条ARM跳转指令，其**跳转目标地址**放在0x001D~0x0003F地址空间，即异常服务函数的入口地址。

所以一个异常向量表实际包含了8个字的异常入口和8个字的跳转目标地址，占用了16个字（64字节）的存储单元。

4.3.5 存储器重映射及引导块

■ ARM异常入口

地址	名称
0x00	该位置被Boot装载程序用作有效用户程序的检测标志。通过定义此保留值，使向量表所有数据32位累加和为0，芯片复位后才能脱机运行用户程序。
0x00	
0x00	
0x00	
0x00	
0x0000 0014	保留*
0x0000 0018	IRQ
0x0000 001C	FIQ

4.3.5 存储器重映射及引导块

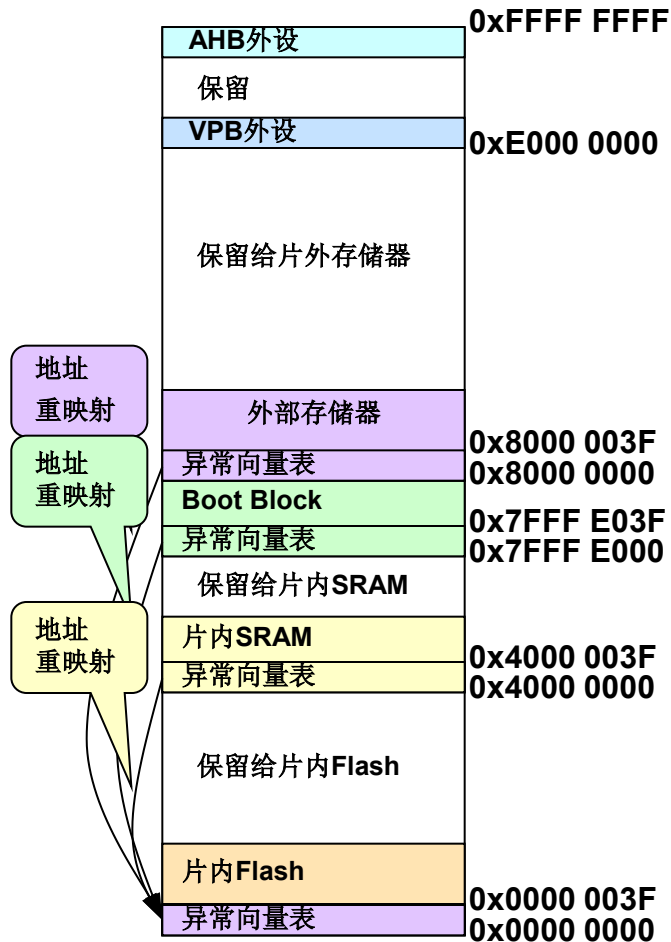
■ 异常向量表的重映射

异常向量表可以来自四个不同的区域：**Boot Block、片内Flash、片内RAM和外部存储器**。微控制器可以执行这些存储器中的代码。

除了片内Flash的向量表位于0x0000~0x003F地址上，其他存储器的向量表都不位于这个地址。为了让ARM内核通过访问0x0000~0x003F地址访问到其他存储区域的向量表，这样向量表必须进行重映射。

注意：除了“用户片内Flash模式”外，其它模式下都无法访问片内Flash的0x0000~0x003F区域。

来自不同区域的异常向量表



复位后，首先运行Boot Block程序，需将Boot Block内0x7FFF E000~0x7FFF E03F的异常向量表重映射到0x0000~0x003F地址以允许处理异常并在Boot装载过程中使用中断。

再根据MEMMAP寄存器的设置运行片外异常向量表。

此时需将片外异常向量表重映射到0x0000~0x003F地址空间。



4.3 存储器寻址

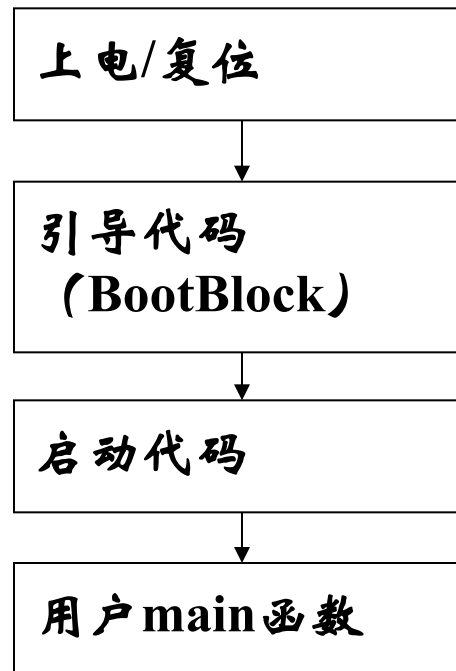
- 1.片内存储器
 - 2.片外存储器
 - 3.存储器映射
 - 4.预取指中止和数据中止异常
 - 5.存储器重映射及引导块
 - 6.启动代码相关部分
-

4.3.6 系统启动代码介绍

• 概述

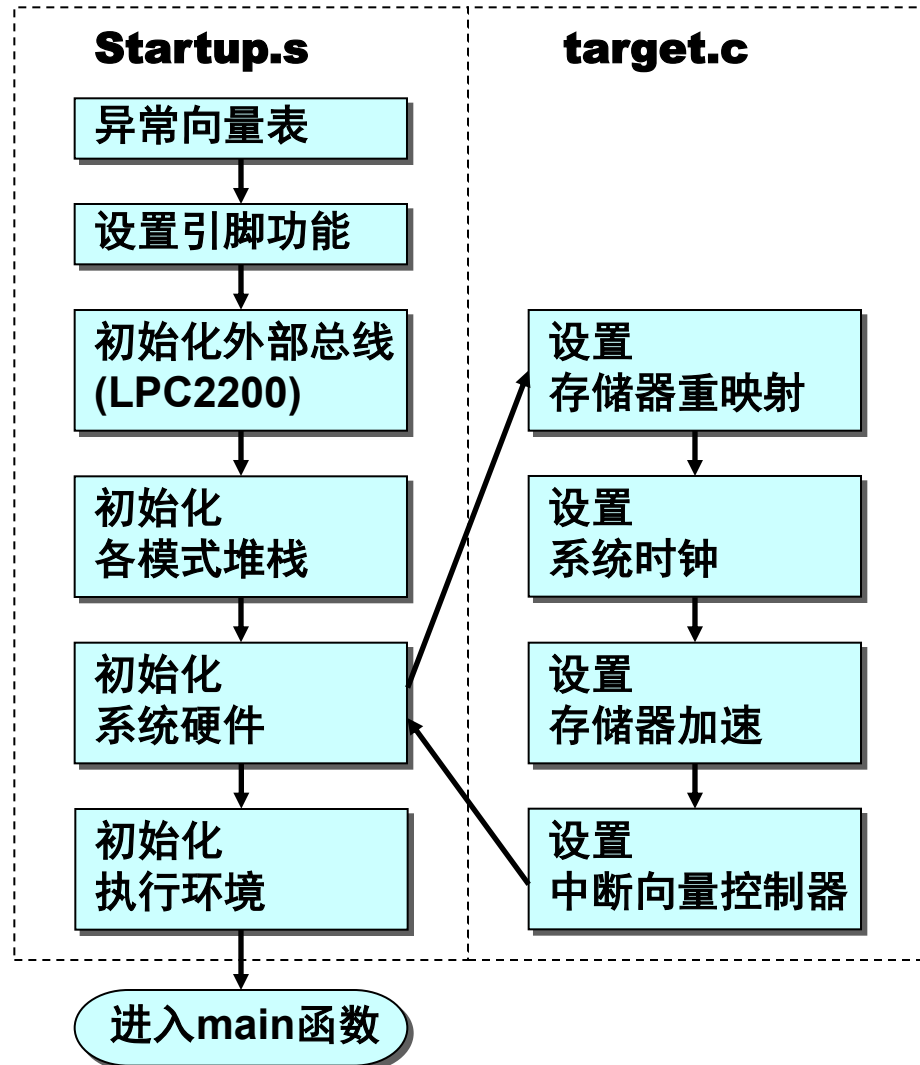
ARM微处理器在上电或复位后首先运行Boot Block中的一段代码，这段代码称为“**引导代码**”，由芯片厂商固化在芯片中。

此后，在正式运行用户main函数之前，还需要运行一段“**启动代码**”，由用户添加。



- 向量表定义;
- 堆栈初始化;
- 系统变量初始化;
- 中断系统初始化;
- I/O初始化;
- 外围初始化;
- 地址重映射等操作。

启动代码流程图





4.4 系统控制模块

- 1.系统控制模块功能汇总
- 2.系统时钟概述
- 3.时钟部件—晶体振荡器
- 4.复位
- 5.时钟部件—唤醒定时器
- 6.时钟部件—PLL(锁相环)
- 7.时钟部件—VPB分频器
- 8.存储器映射控制
- 9.功率控制



4.4.1 系统控制模块功能汇总

概述

在一个ARM系统中，通常有很多功能部件需要某些部件的配合，比如晶体振荡器、状态寄存器、外部复位输入等。这些部件我们统一称之为系统控制模块。

部件名称		功能简介
引脚名称	引脚方向	引脚描述
X1	输入	晶振输入 振荡器和内部时钟发生器电路的输入，使用外部时钟源时，该引脚为时钟输入。
X2	输出	晶振输出 振荡器放大器的输出。
RESET	输入	外部复位输入 该引脚上的低电平将使芯片复位，使I/O口和外设恢复其默认状态，并使处理器从地址0开始执行程序。
功率控制		使处理器空闲或者掉电，还能关闭指定的功能部件，以降低芯片功耗
唤醒定时器		系统上电或掉电唤醒后，保证晶体振荡器能输出稳定的时钟信号

4.4.1 系统控制模块功能汇总

在系统控制模块中，有些部件需要在进行寄存器配置后才能正常工作，如存储器映射控制、锁相环、功率控制、VPB分频器。

名称	描述	访问
存储器映射控制		
MEMMAP	存储器映射控制	R/W
锁相环		
PLLCON	PLL控制寄存器	R/W
PLLCFG	PLL配置寄存器	R/W
PLLSTAT	PLL状态寄存器	RO
PLLFEED	PLL馈送寄存器	WO
功率控制		
PCON	功率控制寄存器	R/W
PCONP	外设功率控制	R/W
VPB分频器		
VPBDIV	VPB分频器控制	R/W

4.4.2 时钟系统

- 概述

时钟是计算机系统的脉搏，处理器核在一拍接一拍的时钟驱动下完成指令执行、状态变换等动作。

外设部件在时钟的驱动下进行着各种工作，比如串口数据的收发、A/D转换、定时器计数等。

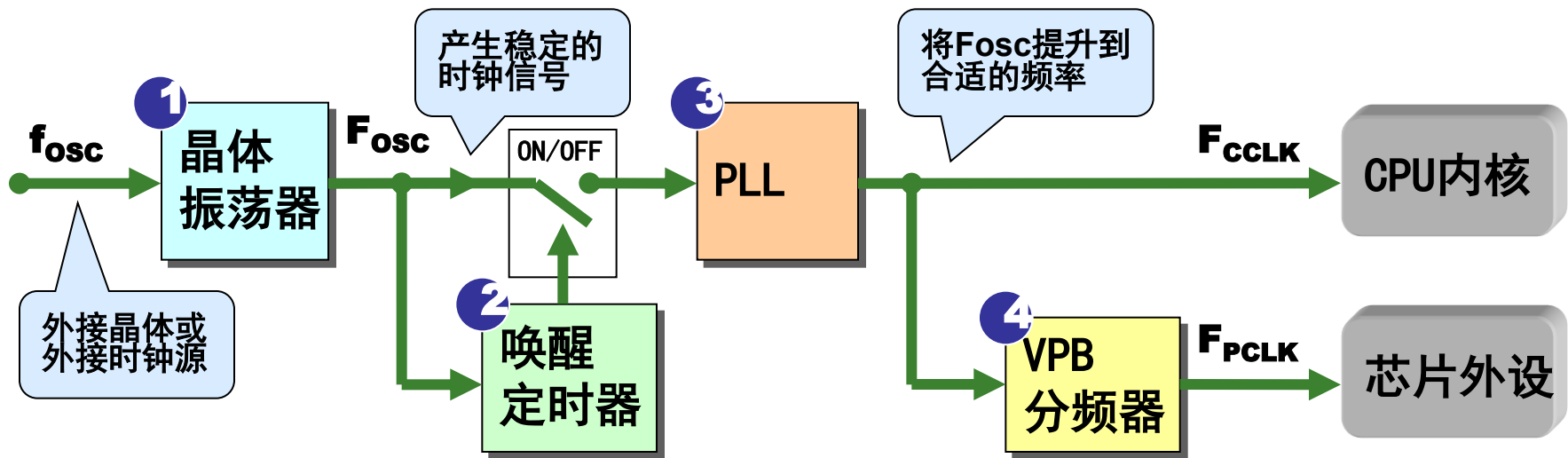
所以时钟对于一个计算机系统是至关重要的，通常时钟系统出现问题也是最致命的，比如振荡器不起振、振荡不稳、停振等。

4.4.2 时钟系统

时钟系统结构

LPC2000系列微控制器的时钟系统包括四个部分：**晶体振荡器**、**唤醒定时器**、**锁相环（PLL）**和**VPB分频器**。

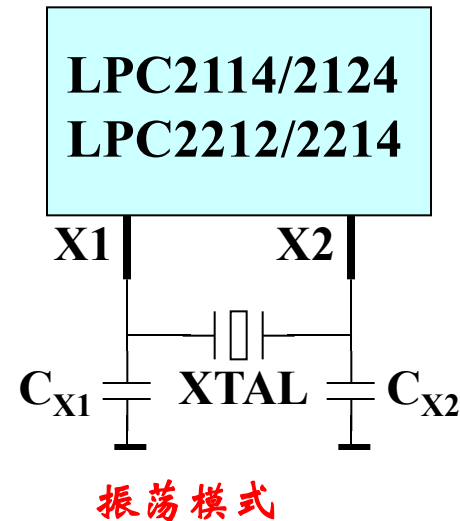
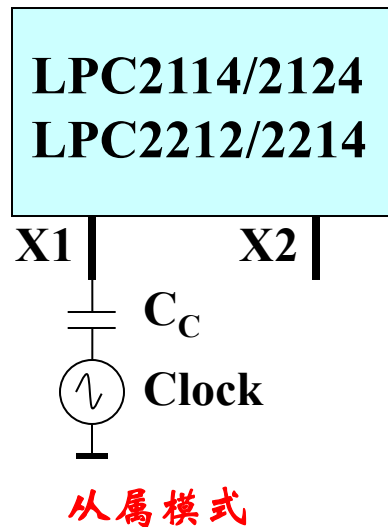
晶体振荡器为系统提供基本的时钟信号（F_{OSC}）。唤醒定时器可以在低功耗模式下产生稳定的时钟信号（F_{OSC}）。PLL将F_{OSC}提升到合适的频率。VPB分频器将F_{OSC}降低到合适的频率。CPU内核和外设都需要时钟信号进行初始化。



4.4.3 时钟部件—晶体振荡器

• 概述

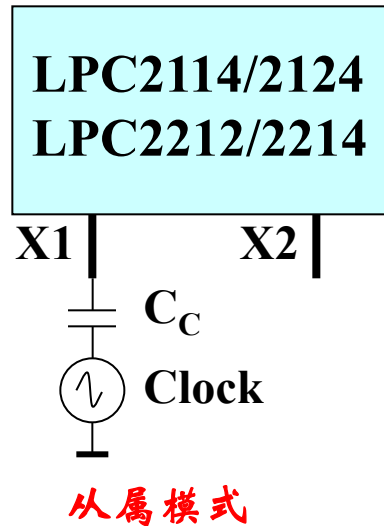
LPC2000系列微控制器的晶体振荡器可以使用**外部时钟源**(从属模式), 也可以使用**外接晶体和片内振荡电路**(振荡模式)产生时钟。



4.4.3 时钟部件 — 晶体振荡器

- 从属模式

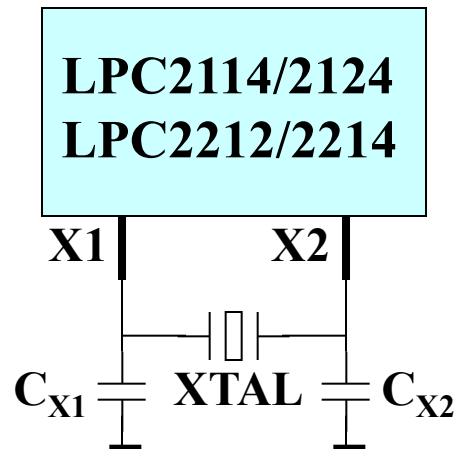
使用从属模式时，时钟信号通过X1引脚从外部输入，输入频率范围：**1~50(MHz)**，其幅度范围为：**200mV~1.8V**。



4.4.3 时钟部件—晶体振荡器

- 振荡模式

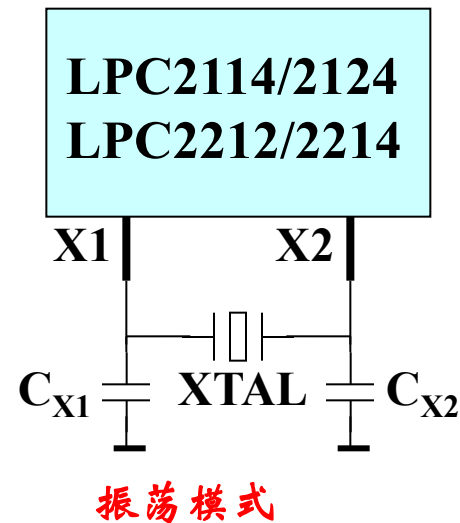
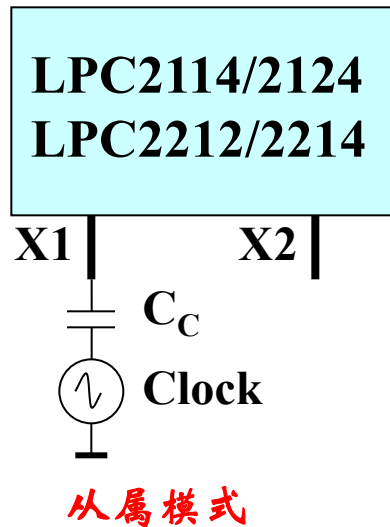
使用振荡模式时，时钟信号由内部晶体振荡器和外部连接的晶体振荡产生，振荡频率范围：**1~30(MHz)**。



振荡模式

4.4.3 时钟部件—晶体振荡器

- 注意：**如果使用了ISP下载功能或者连接PLL提高频率，则输入的时钟频率范围必须在**10~25 (MHz)**之间。





4.4.4 复位

• 概述

复位指将计算机系统中的硬件逻辑归位到一个初始的状态，比如让寄存器恢复默认值、让处理器从第一条指令开始执行程序等。

LPC2000系列芯片有两个复位源：

1、外部复位

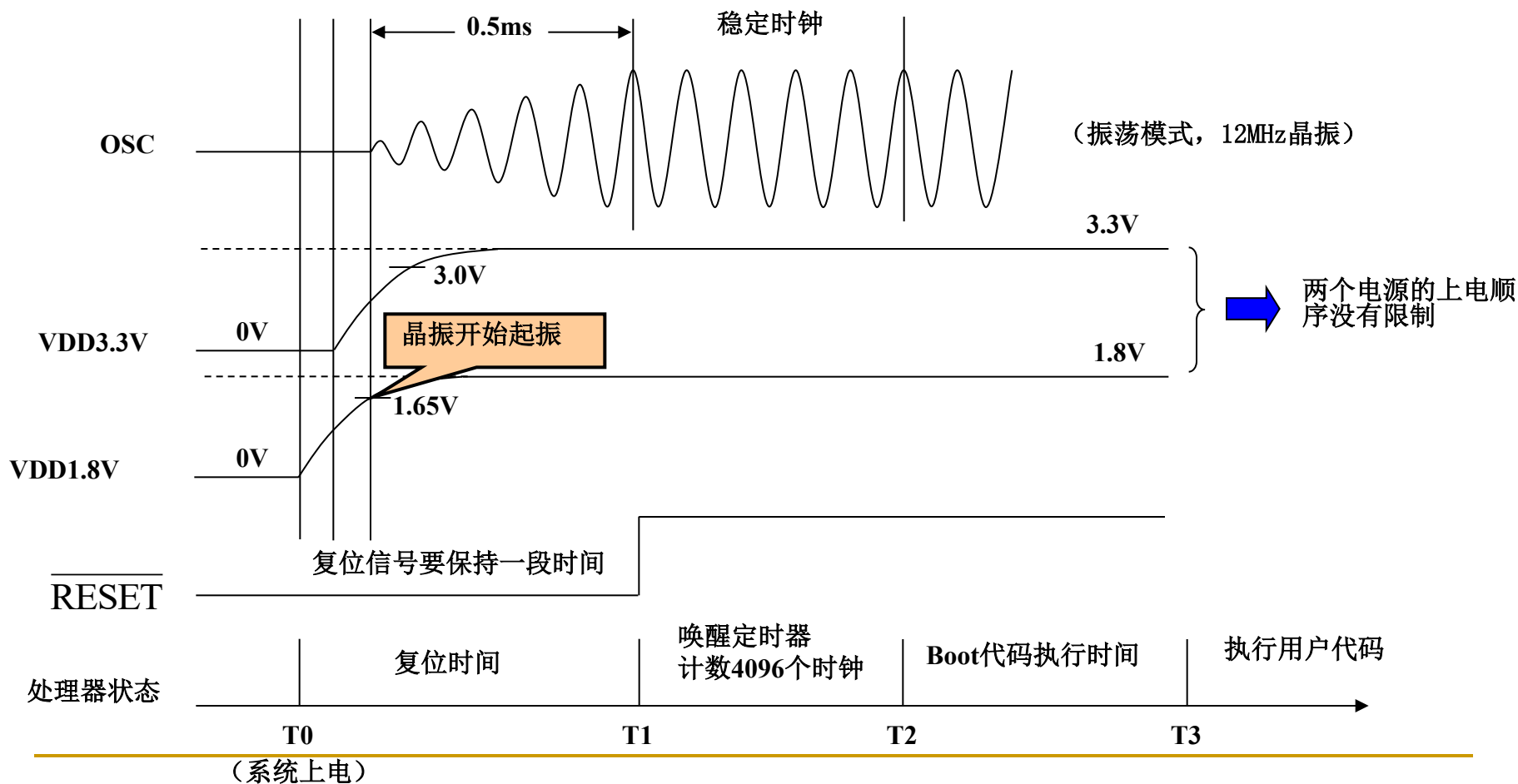
—把nRESET引脚拉为低电平，并保持一个最小时间，引发复位

2、看门狗复位

—通过设置看门狗相关寄存器，当看门狗定时器溢出后，引发复位

4.4.4 复位

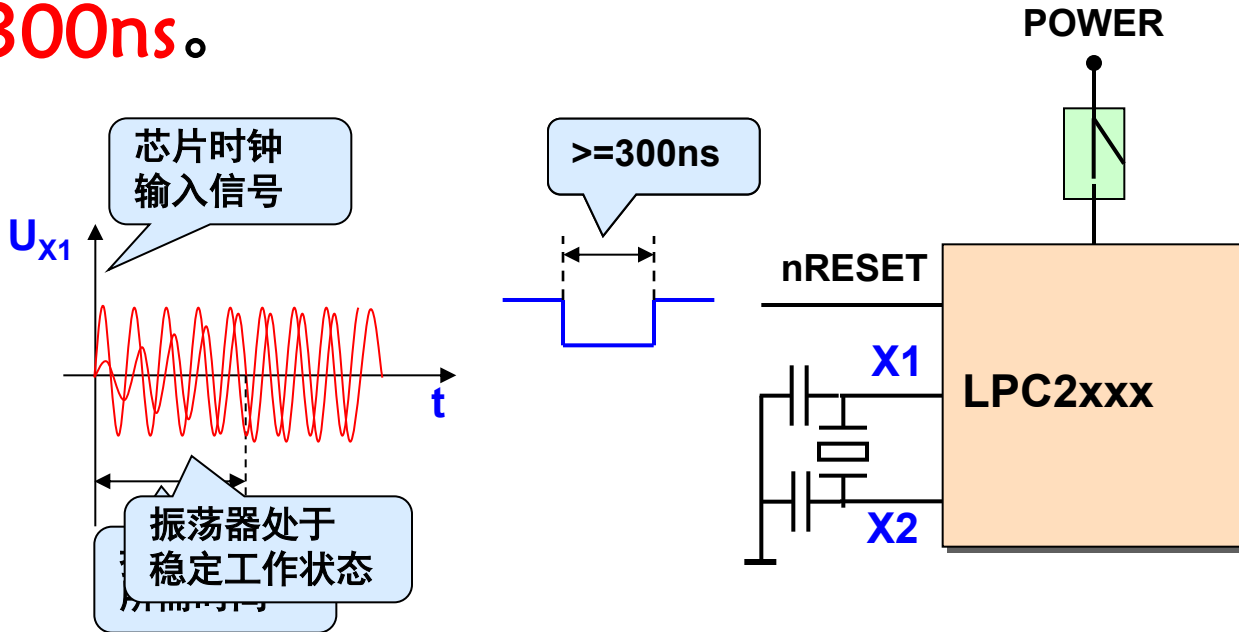
■ 硬件复位流程



4.4.4 复位

外部复位

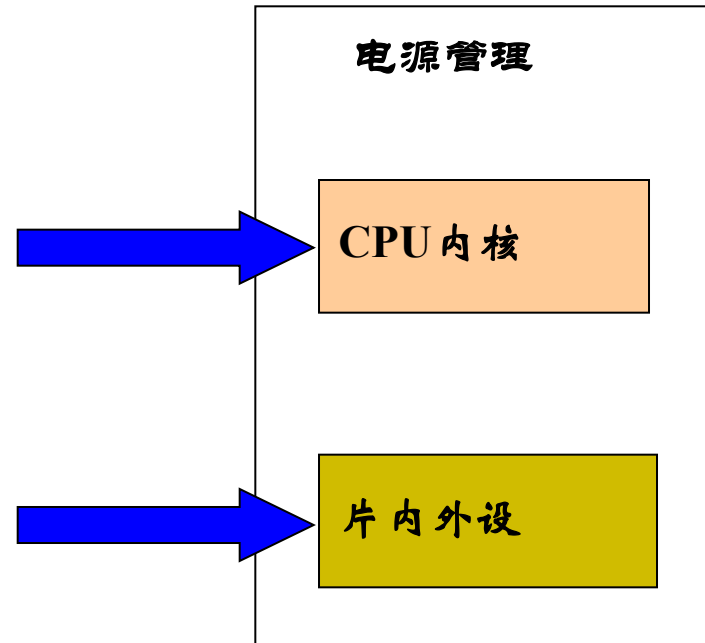
③ 微控制器上电后，内部寄存器未初始化，系统未运行，开始到稳定需要一段时间，所以外部复位信号至少要有保持10ms，外部复位信号可以缩短到不小于300ns。



4.4.4 复位

• 复位与电源上电次序

- V_{18} : 数字1.8V供电电源
- V_{18A} : 模拟1.8V供电电源
- V_3 : 数字3.3V供电电源
- V_{3A} : 模拟3.3V供电电源



1.8V为内核供电，因此1.8V电源必须正常上电。

4.4.4 复位

• 外部复位和内部WDT复位的区别

外部复位

判断引脚:

- **P1.20 /TRACESYNC**
- **P1.26/RTCK**
- **BOOT1和BOOT0**

判断引脚: **P0.14**

执行用户程序
或
运行ISP程序

时间

T0

T1

T2

WDT复位

执行用户程序
或
运行ISP程序

时间

T0



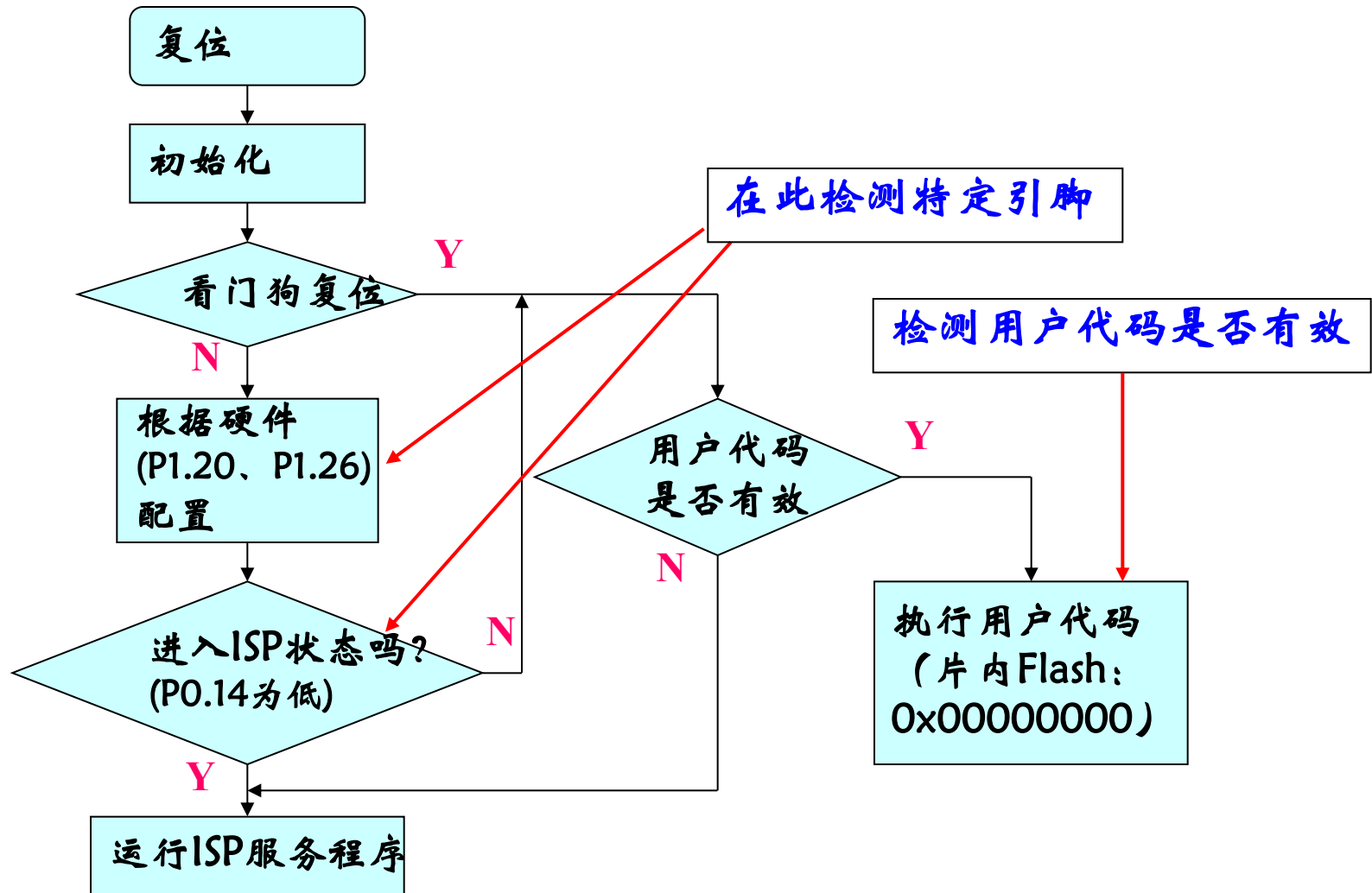
4.4.4 复位

- **复位与Boot Block**

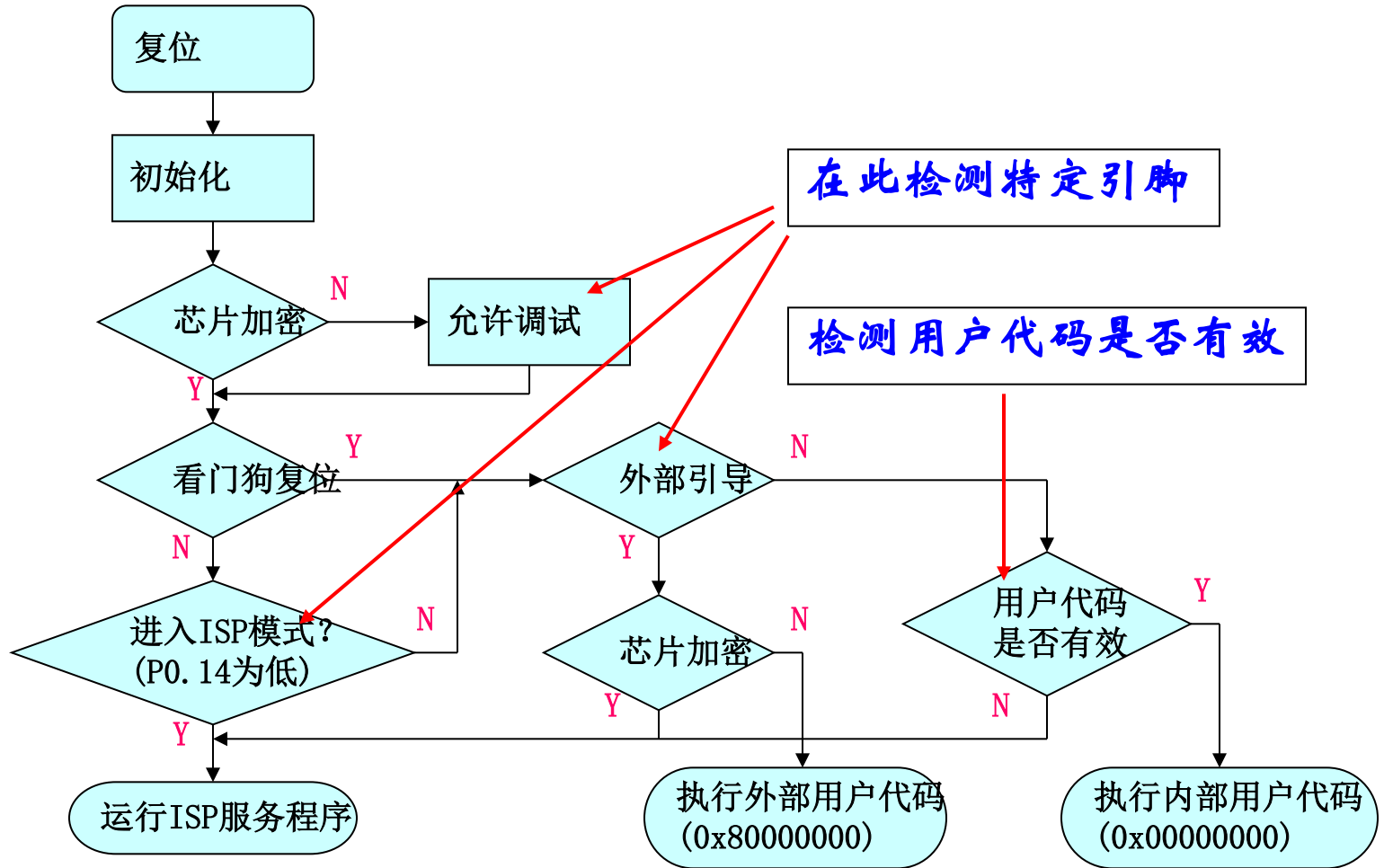
Boot Block是芯片生成时由厂家固化在其中的一段代码，用户无法修改或删除，这段代码在复位后被首先运行。

注：不同芯片的Boot Block功能也不尽相同。

■ Boot程序工作流程—LPC2114/2124



■ Boot程序工作流程—LPC2210/2212/2214



4.4.4 复位

• 复位与Boot Block

Boot Block的功能包括：

 运行哪个存储器上的程序。

LPC2200系列微控制器可以同时存在片内存储器和片外存储器，Boot Block通过芯片上的BOOT0和BOOT1引脚来判断程序的运行。

注：LPC2100系列微控制器只有片内Flash，它们无需判断。

P2.27/D27/BOOT1	P2.26/D26/BOOT0	引导方式
0	0	CS0控制的8位存储器
0	1	CS0控制的16位存储器
1	0	CS0控制的32位存储器
1	1	内部Flash存储器

4.4.4 复位

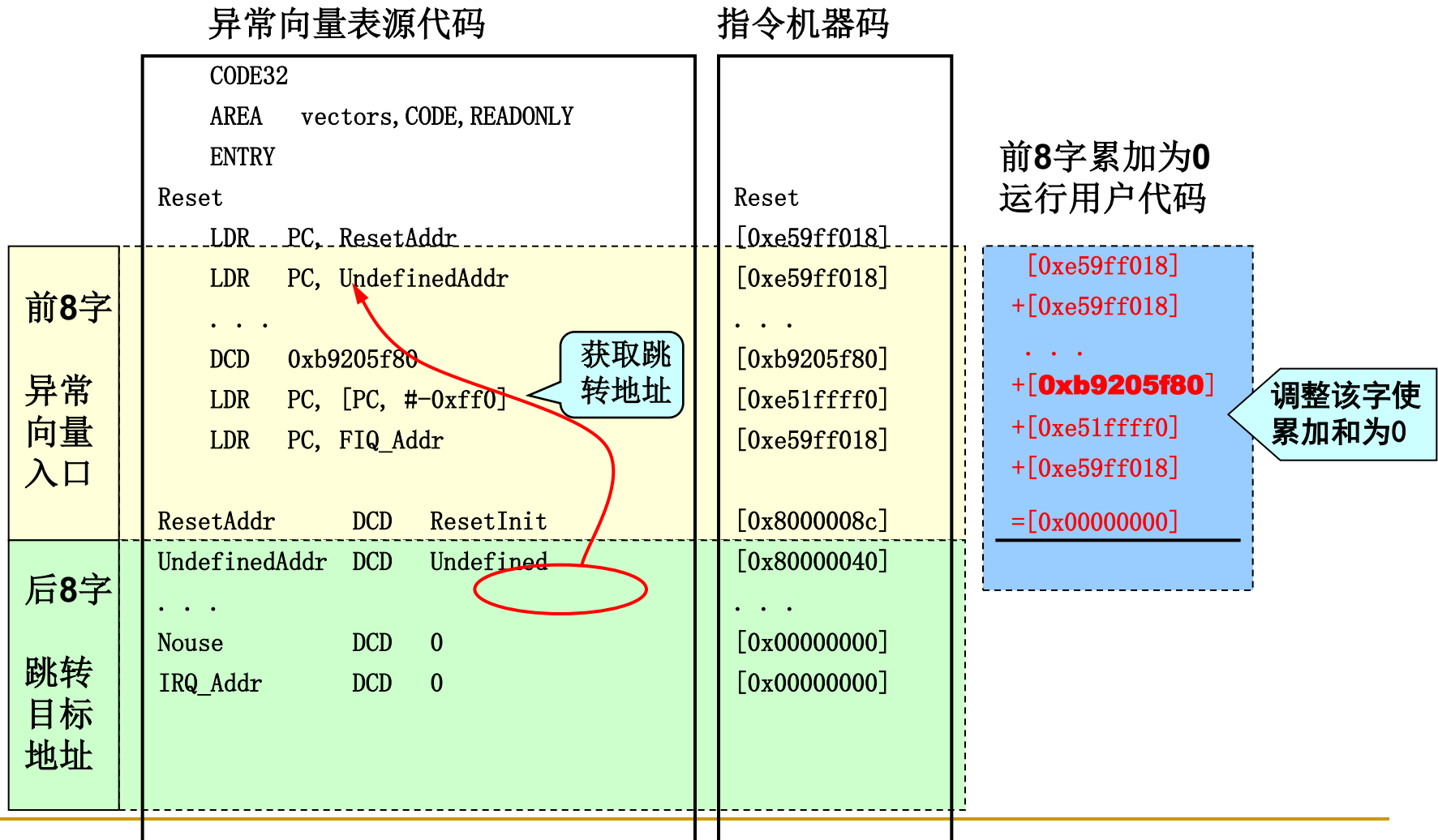
- 复位与Boot Block

Boot Block的功能包括：

- 用户代码是否有效。

Boot Block在将芯片的控制权交给用户程序之前，要先判断用户程序是否有效，否则将不运行用户程序，这样可以避免在现场设备中的芯片因为代码损坏而导致程序乱飞引起事故的情况发生。

用户代码有效判别方案



4.4.4 复位

• 复位与Boot Block

Boot Block的功能包括：

☾ 芯片是否加密。

芯片可加密可以保护芯片用户的知识产权不受侵害。

加密后：

➤ 芯片禁止JTAG接口调试；

➤ 芯片限制ISP命令，只能执行芯片整片擦除。

对芯片加密的方法：在芯片Flash的0x01FC地址处写入数据0x87654321即可。当Boot Block检测到该地址存在加密标志字时，就对芯片的JTAG和ISP操作进行限制，达到加密的效果。

4.4.4 复位

• 复位与Boot Block

Boot Block的功能包括：

🕒 在应用编程(IAP)。

LPC2000系列微控制器内部的Flash是无法从外部直接擦写的，这些功能必须通过IAP代码来实现。IAP可以实现片内Flash的擦除、查空、将数据从RAM写入指定Flash空间、校验、读器件ID等功能。

应用实例：

- 数据存储
- 在线升级

4.4.4 复位

• 复位与Boot Block

Boot Block的功能包括：

⑤ 在系统编程(ISP)。

ISP功能是一种非常有用的片内Flash烧写方式。ISP工作时，通过UART0使用约定协议与计算机上的ISP软件进行通信，并按用户的操作要求，调用内部的IAP代码实现各种功能。比如把用户代码下载到片内Flash中。

有两种情况可以使芯片进入ISP状态

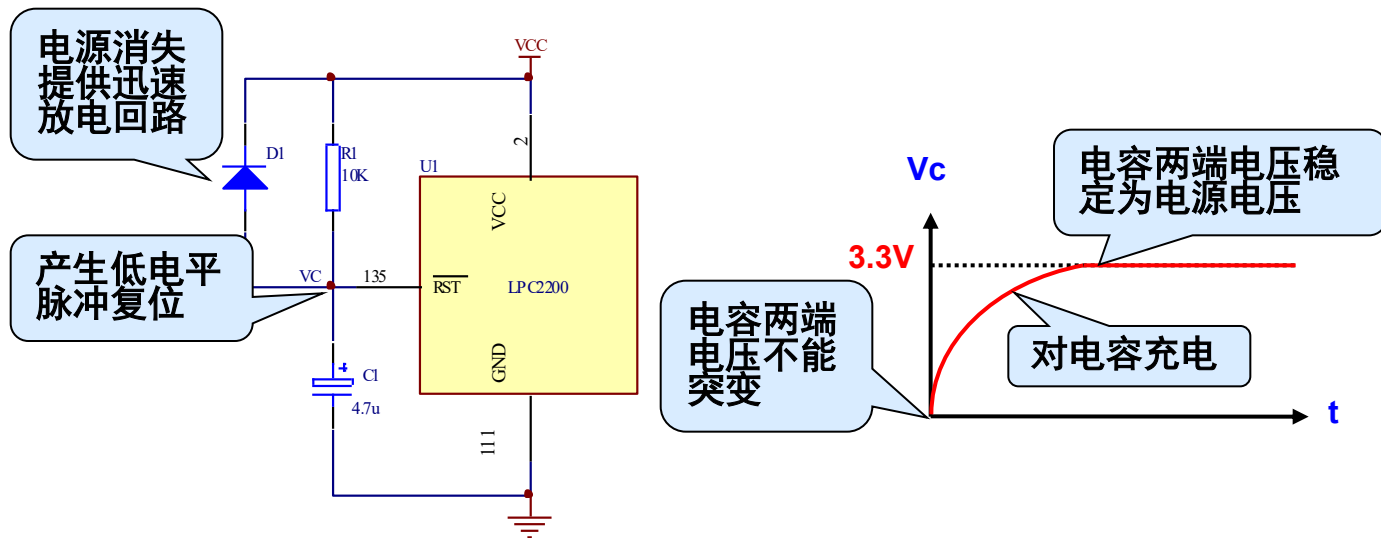
- 1、将芯片的P0.14引脚拉低后，复位芯片可以进入ISP状态；**
- 2、在芯片内部无有效用户代码时，Boot Block进入ISP状态。**

4.4.4 复位

复位及复位芯片配置

这些微控制器低廉在但电路性能佳性价比产生稳定可靠微控制器需要所部输般透舍需要使用CAT809、SP708和CAT1025等专门的复位芯片。

阻容式复位电路：

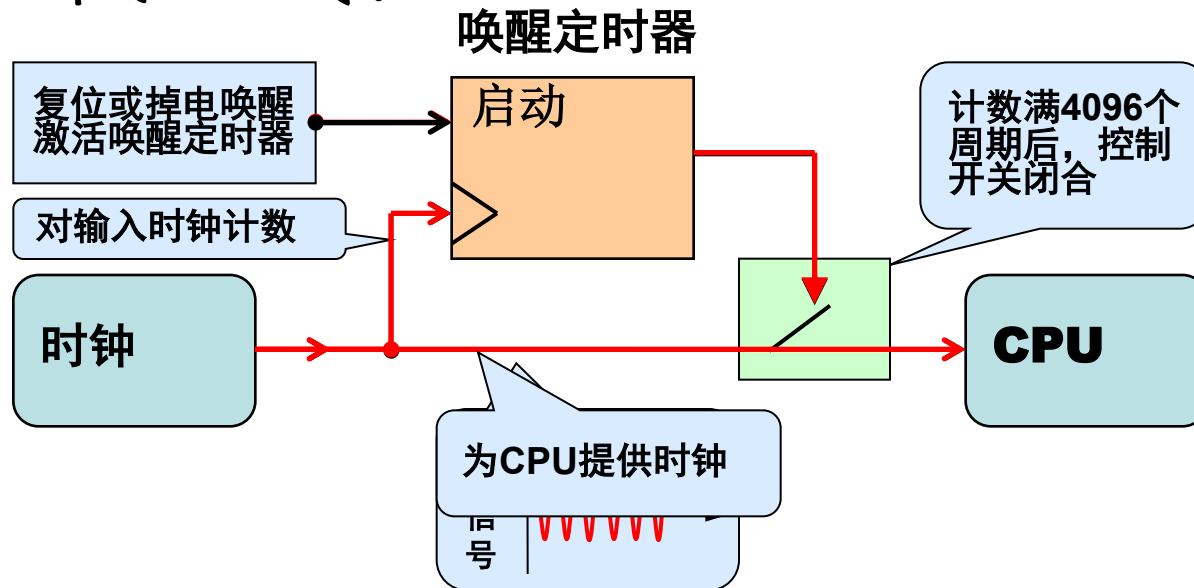


4.4.5 时钟部件—唤醒定时器

• 概述

唤醒定时器能够确保振荡器和芯片内部硬件电路在处理器开始执行指令之前有足够的时间初始化。

工作原理如图：

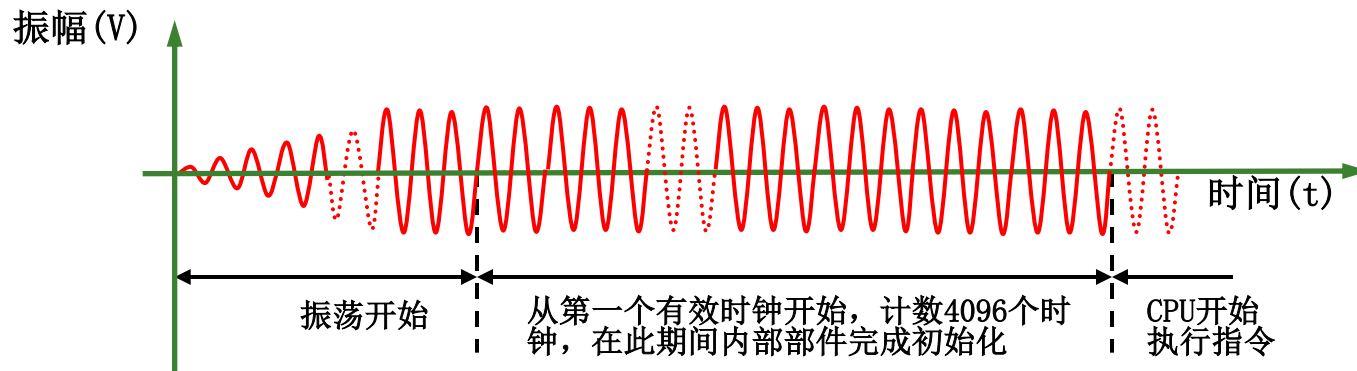


4.4.5 时钟部件—唤醒定时器

• 概述

当给芯片加电或某个事件使芯片退出掉电模式后，振荡器就开始工作，但是需要一段时间来产生足够振幅的信号驱动时钟逻辑。振荡的波形大致如下：

注：唤醒定时器就通过监测晶振状态来判断是否能开始可靠的执行代码。



4.4.5 时钟部件—唤醒定时器

- 唤醒定时器与时钟的关系

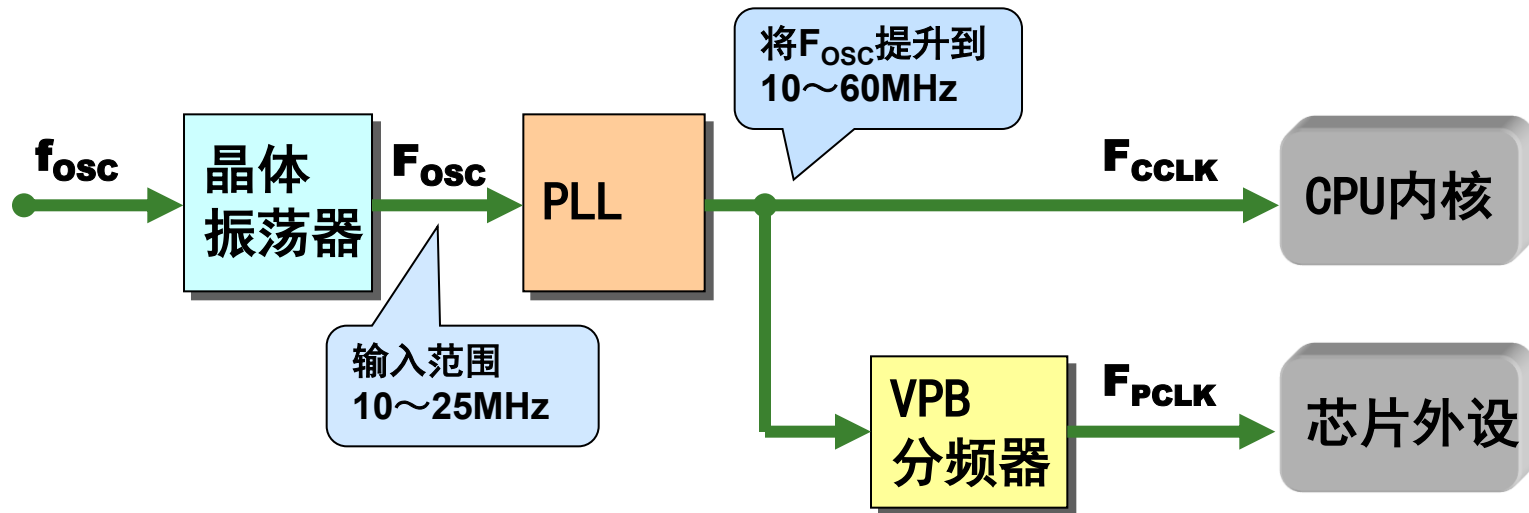
唤醒定时器检测到有效时钟信号后，计数4096个时钟脉冲，并在这段时间里初始化系统硬件。如芯片满足运行条件(Flash初始化完成、外部复位信号已撤除等)，接通系统时钟，处理器开始执行指令。

总之，LPC2000系列芯片的唤醒定时器是根据晶振的情况来执行最短时间的复位，它在处理器从掉电模式中唤醒或发生了任何复位时激活。

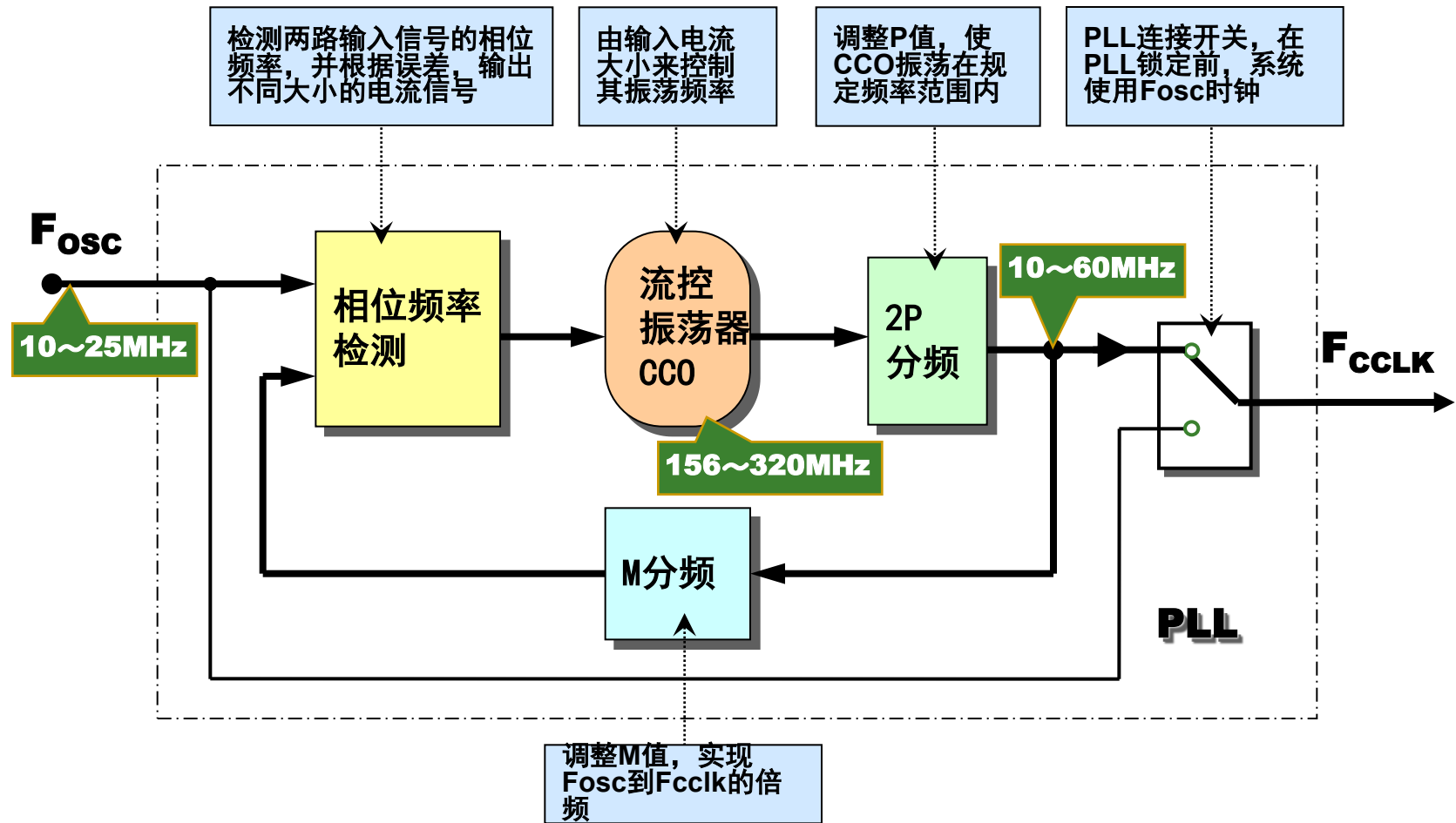
4.4.6 时钟部件—PLL(锁相环)

- 概述

LPC2000系列芯片内部均具有PLL电路，振荡器产生的时钟 F_{osc} 通过PLL升频，可以获得更高的系统时钟(F_{cclk})。



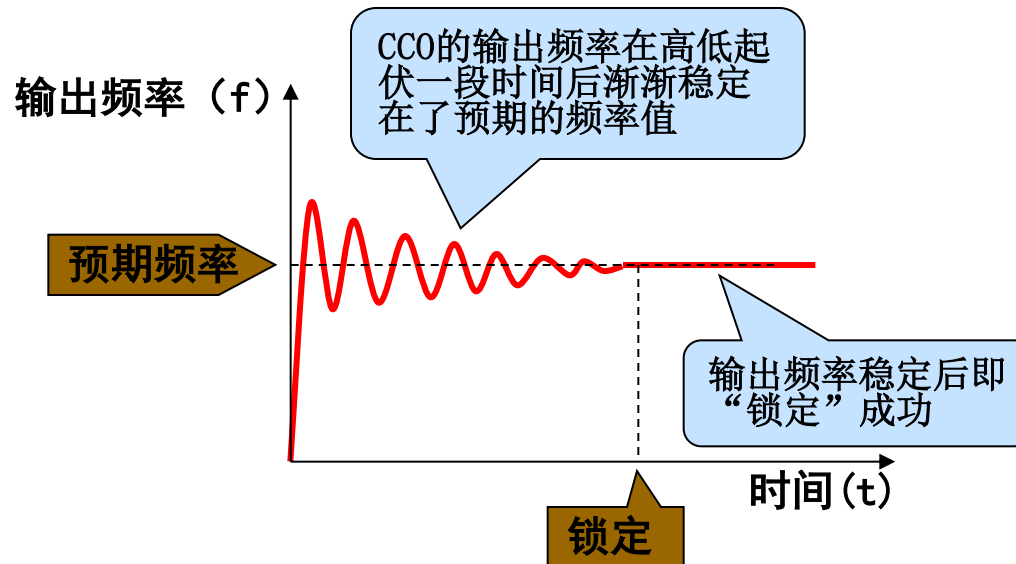
PLL内部结构框图



4.4.6 时钟部件—PLL(锁相环)

• PLL的锁定过程

CCO的输出频率受到“相位频率检测”部件的控制，输出所需频率的过程不是一蹴而就的，而是一个拉锯反复的过程。





4.4.6 时钟部件—PLL(锁相环)

• 寄存器描述

与PLL相关的寄存器有四个，其中三个为控制寄存器，还有一个是状态寄存器。

地址	名称	描述	访问
0xE01FC080	PLLCON	PLL控制寄存器。最新的PLL控制位的保持寄存器。写入该寄存器的值在有效的PLL馈送序列执行之前不起作用。	R/W
0xE01FC084	PLLCFG	PLL配置寄存器。最新的PLL配置值的保持寄存器。写入该寄存器的值在有效的PLL馈送序列执行之前不起作用。	R/W
0xE01FC088	PLLSTAT	PLL状态寄存器。PLL控制和配置信息的读回寄存器。如果曾对PLLCON或PLLCFG执行了写操作，但没有产生PLL馈送序列，这些值将不会反映PLL的当前状态。读取该寄存器提供了控制PLL和PLL状态的真实值。	RO
0xE01FC08C	PLLFEED	PLL馈送寄存器。该寄存器使能装载PLL控制和配置信息，该配置信息从PLLCON和PLLCFG寄存器装入实际影响PLL操作的映像寄存器。	WO

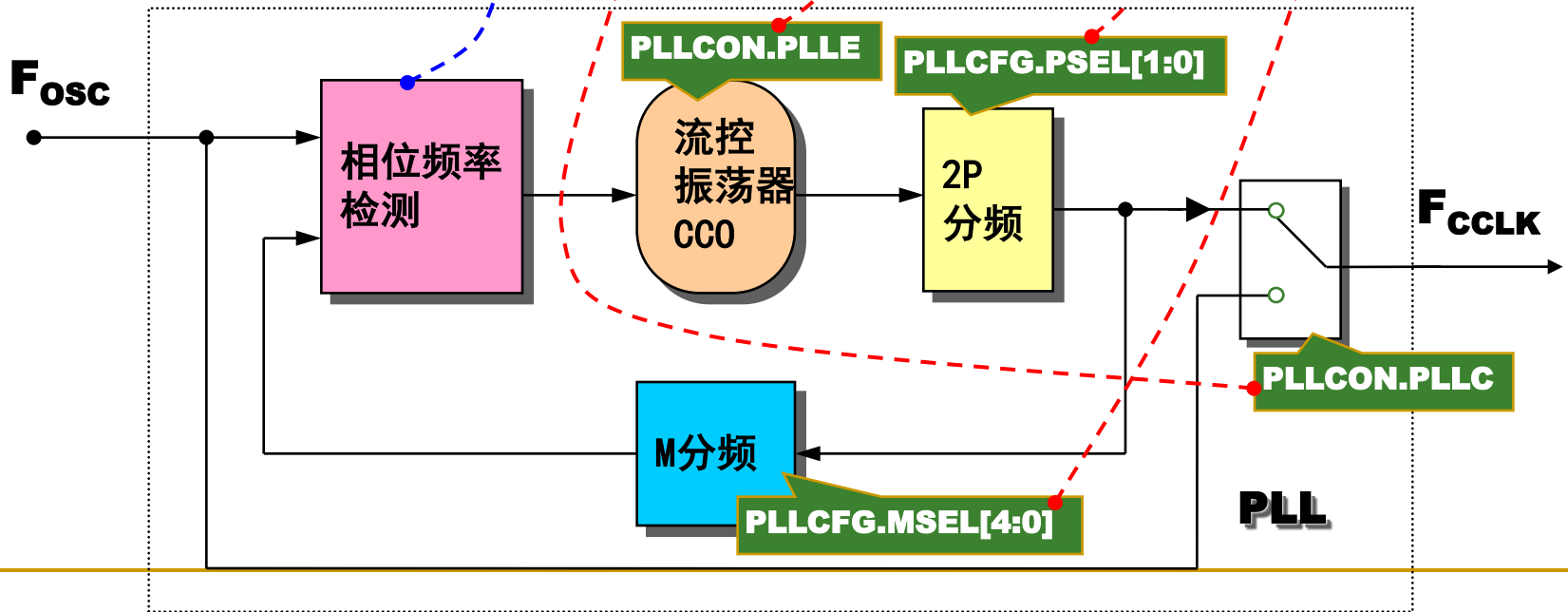
寄存器描述

PLL状态寄存器(PLLSTAT):

PLL控制寄存器(PLLCON):

MSEL[4:0]、PSEL[1:0]、PLLE、PLLCL: 读出反映这几个参数的设置值, 写入无效:

PLLCL	PLLE	PLL功能
0	0	PLL被关闭, 并断开连接。
0	1	PLL被激活但是尚未连接。可以在PLOCK置位后连接。
1	0	与00组合相同。避免PLL已连接, 当还没有使能的情况。
1	1	PLL已经使能, 并连接到处理器作为系统时钟源。



寄存器描述

PLL馈送寄存器(PLLFEED):

PLLFEED[7:0]: PLL馈送序列必须写入该寄存器才能使PLL配置和控制寄存器的更改生效。

馈送序列分两步进行, 如下所示:

step1. 将值**0xAA**写入PLLFEED

step2. 将值**0x55**写入PLLFEED

馈送寄存器 PLLFEED	7 : 0
	PLLFEED[7 : 0]

操作示例

```
DISABLE_IRQ();           //关闭中断, 防止馈送序列操作被打断
PLLFEED=0xAA;            //馈送序列第一步
PLLFEED=0x55;            //馈送序列第二步
ENABLE_IRQ();             //馈送序列操作结束, 打开中断
```

注: 这两个写操作的顺序必须正确, 且必须是连续的VPB总线周期。



4.4.6 时钟部件—PLL(锁相环)

- PLL和掉电模式

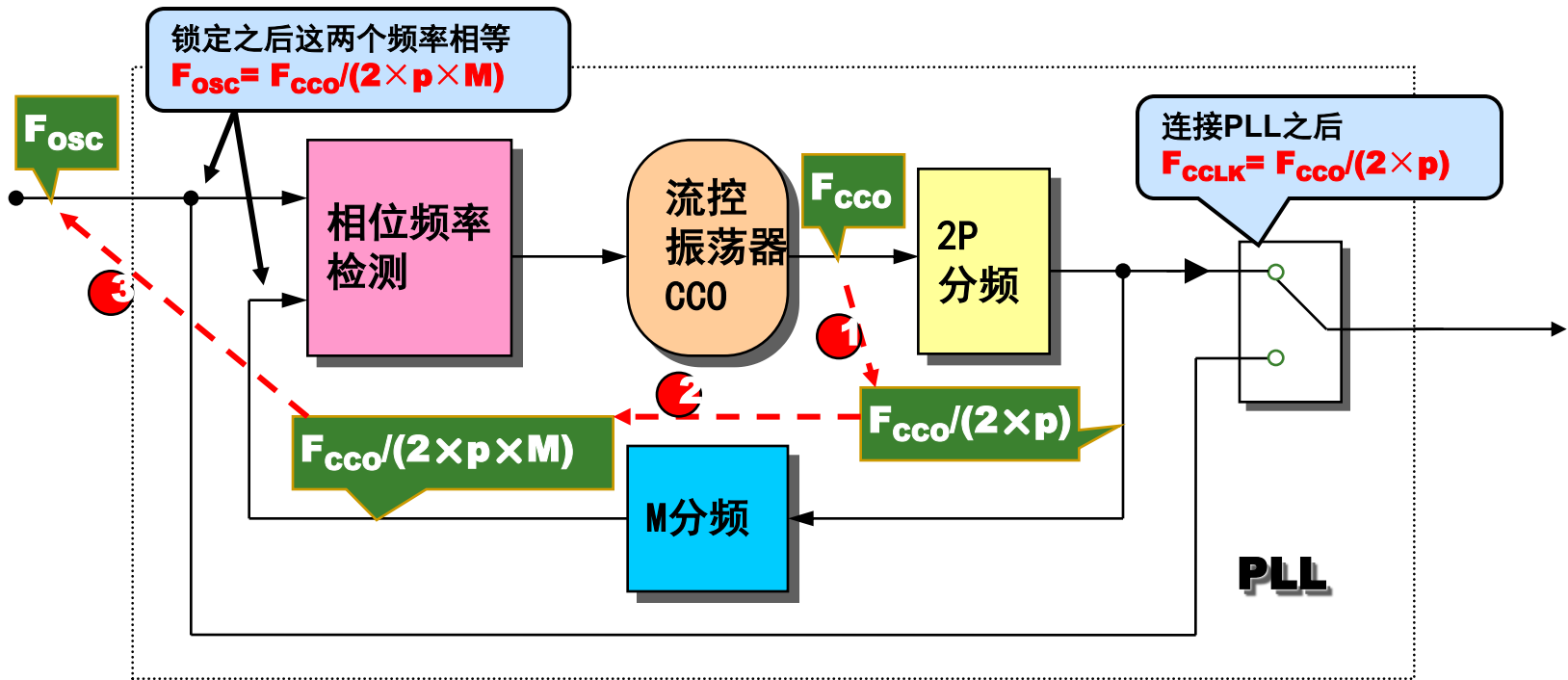
掉电模式会自动关闭并断开PLL。从掉电模式唤醒不会自动恢复PLL的设定，PLL的恢复必须由软件来完成。通常，首先将PLL激活并等待锁定，然后再将PLL连接。

注：不要试图在掉电唤醒之后简单地执行馈送序列来重新启动PLL，因为这会在PLL锁定建立之前同时使能并连接PLL。

4.4.6 时钟部件—PLL(锁相环)

• PLL频率计算

$F_{cco}/(2 \times p)$ 信号经过“M分频”部件，得到 $F_{cco}/(2 \times p \times M)$ 的频率，该频率与晶体振荡器频率相等，即 $F_{osc} = F_{cco}/(2 \times p \times M)$





4.4.6 时钟部件—PLL(锁相环)

- PLL频率计算

可以得出以下几个等式:

$$F_{osc} = F_{cco} / (2 \times p \times M) \rightarrow F_{cco} = F_{osc} \times (2 \times p \times M)$$

$$F_{cclk} = F_{cco} / (2 \times p) \rightarrow F_{cco} = F_{cclk} \times (2 \times p)$$

最后得出PLL的输出频率(当PLL激活并连接时)为:

$$F_{cclk} = M \times F_{osc} \quad \text{或} \quad F_{cclk} = F_{cco} / (2 \times P)$$

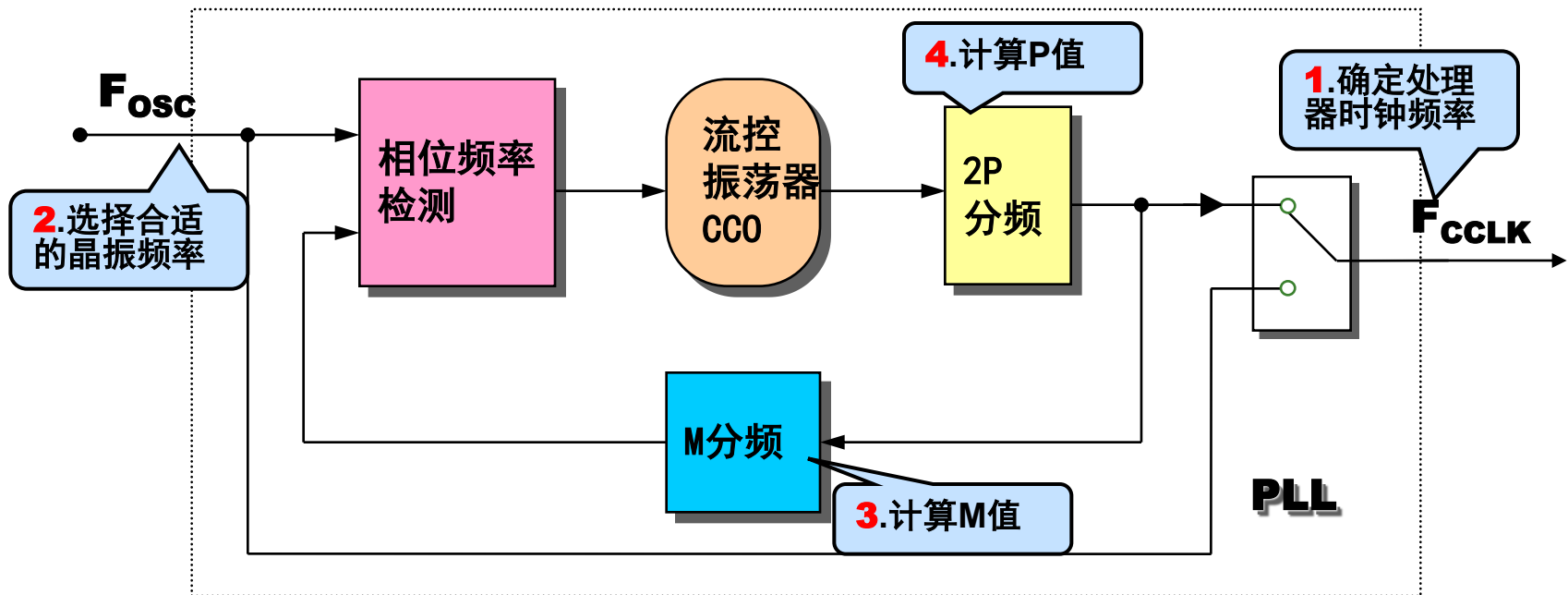
CCO输出频率为:

$$F_{cco} = F_{cclk} \times 2 \times P \quad \text{或} \quad F_{cco} = F_{osc} \times M \times 2 \times P$$

4.4.6 时钟部件—PLL(锁相环)

确定PLL设定的过程

选择晶振频率(F_{OSC})。F_{CLK}一定是F_{OSC}的整数倍。
 选择晶振频率(F_{OSC})。F_{CLK}一定是F_{OSC}的整数倍。
 选择晶振频率(F_{OSC})。F_{CLK}一定是F_{OSC}的整数倍。



• PLL设置举例

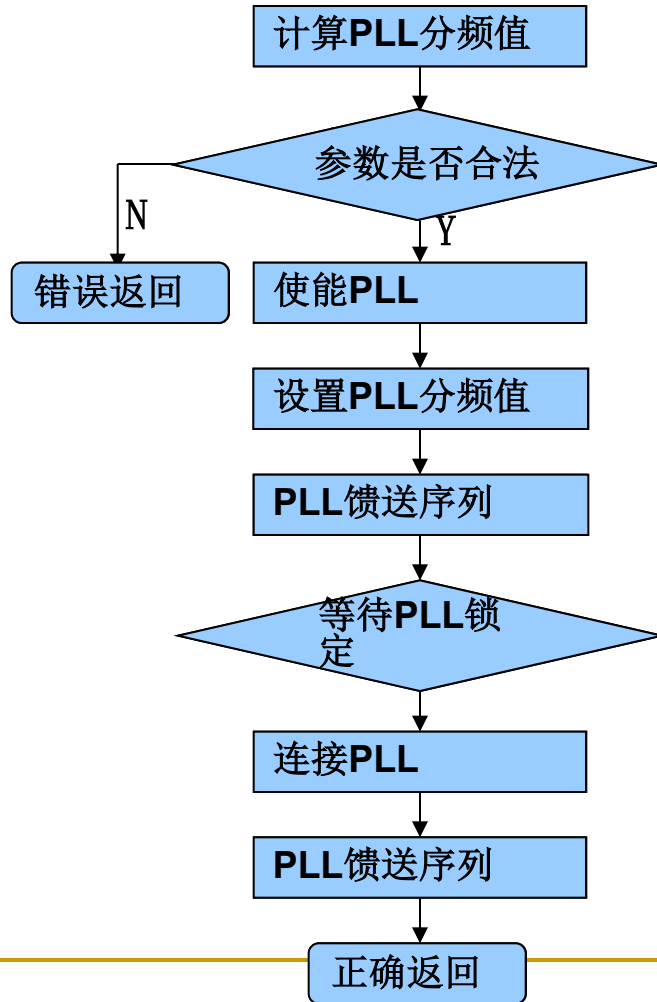
系统要求 $F_{osc} = 10\text{MHz}$, $F_{cclk} = 60\text{MHz}$ 。

根据这些要求：

1. 确定 $F_{cclk} = 60\text{MHz}$;
2. 选择 $F_{osc} = 10\text{MHz}$;
3. 计算 $M = F_{cclk}/F_{osc} = 60\text{MHz}/10\text{MHz} = 6$ 。 $M-1 = 5$,
所以写入 $\text{PLLCFG}[4:0] = 00101$;
4. 计算 $P = F_{cco}/(F_{cclk} * 2)$, 其中 F_{cco} 为 $156 \sim 320\text{MHz}$ 。
当 $F_{cco} = 156\text{MHz}$ 时, $P = 156\text{MHz}/(2 * 60\text{MHz}) = 1.3$
当 $F_{cco} = 320\text{MHz}$ 时, $P = 2.67$
 P 取整数 2, 所以写入 $\text{PLLCFG}[6:5] = 01$ 。

PLL设置举例

PLL配置过程：



```

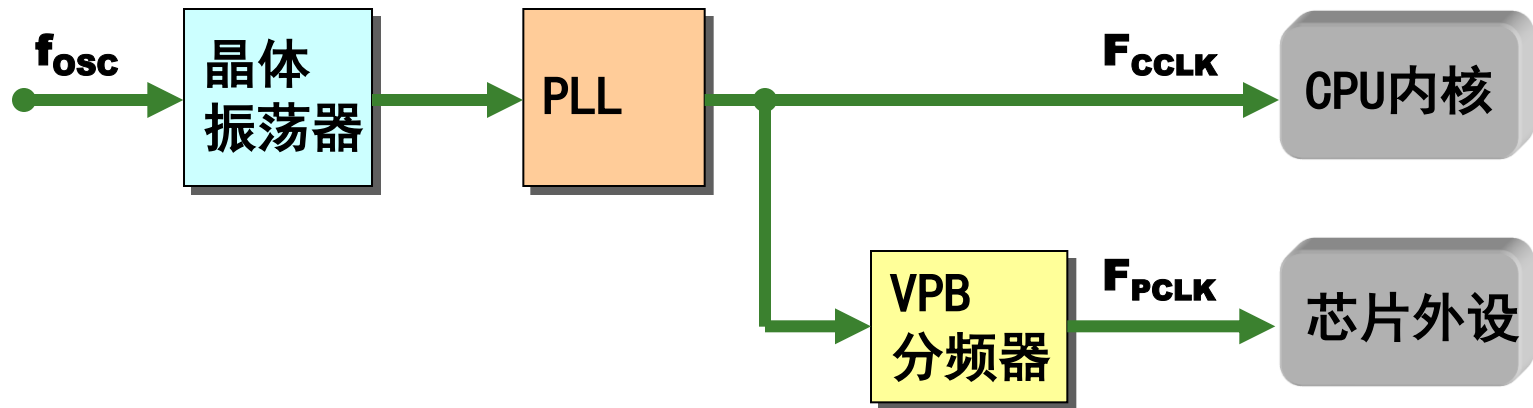
uint8 PLLSet(uint32 Fcclk, uint32 Fosc, uint32
Fcco)
{
    uint8 i;
    uint32 plldat;
    i = (Fcco / Fcclk);
    switch(i)
    {
        case 2:
            plldat = ((Fcclk / Fosc) - 1) | (0 << 5);
            break;
        case 4:
            plldat = ((Fcclk / Fosc) - 1) | (1 << 5);
            break;
        case 8:
            plldat = ((Fcclk / Fosc) - 1) | (2 << 5);
            break;
        case 16:
            plldat = ((Fcclk / Fosc) - 1) | (3 << 5);
            break;
        default:
            return(FALSE); break;
    }
    PLLCON = 1;
    PLLCFG = plldat;
    PLLFEED = 0xaa;
    PLLFEED = 0x55;
    while((PLLSTAT & (1 << 10)) == 0);
    PLLCON = 3;
    PLLFEED = 0xaa;
    PLLFEED = 0x55;
    return(TRUE);
}
  
```

4.4.7 时钟部件—VPB分频器

概述

VPB分频器是系统内部总线：上面挂接着绝大部分的将处理器与外设的工作频率，相对于ARM内核来说都要慢工牌。VPB分频器决定了处理器时钟（cclk）与外设器件所使用的时钟（pclk）之间的关系。

1. 降低系统功耗。





4.4.7 时钟部件—VPB分频器

• VPBDIV寄存器

VPBDIV[1:0]: 设置分频值，可以设定3个值；

VPBDIV[1:0]	说明
00	VPB总线时钟为处理器时钟的1/4。
01	VPB总线时钟与处理器时钟相同。
10	VPB总线时钟为处理器时钟的1/2。
11	保留。写入该值将不改变分频值。

XCLKDIV[1:0]: 这些位用于控制LPC2200系列微控制器A23/XCLK引脚上的时钟驱动，编码方式与VPBDIV相同。

控制寄存器 VPBDIV	7 : 6	5 : 4	3 : 2	1 : 0
	—	XCLKDIV[1:0]	—	VPBDIV[1:0]

4.4.7 时钟部件—VPB分频器

• VPBDIV设置示例

根据外设时钟与内核时钟的关系，
设置对应的VPB值：

- 外设时钟是内核时钟相等频。

```
void VPBSet(uint32 Fcclk, uint32 Fpclk)
{
    uint8 i;
    if((Fpclk / (Fcclk / 4)) == 1)
    {
        VPBDIV = 0;
    }
    else
    if((Fpclk / (Fcclk / 4)) == 2)
    {
        VPBDIV = 2;
    }
    else
    if((Fpclk / (Fcclk / 4)) == 4)
    {
        VPBDIV = 1
    }
}
```




4.4.8 存储器映射控制

- 概述

为了允许运行在不同存储器空间中的代码对中断进行控制，需要使用存储器映射控制机制改变地址0x0000~0x003F的中断向量的映射。

4.4.8 存储器映射控制

• 存储器映射控制寄存器

存储器映射控制寄存器 (MEMMAP) 是一个可读可写的寄存器。

控制寄存器 MEMMAP	7 : 2	1 : 0
	—	MAP[1:0]

异常向量表可以重新映射到：

- Boot Block;
- 片内Flash;
- 片内SRAM;
- 外部存储器Bank0。

4.4.8 存储器映射控制

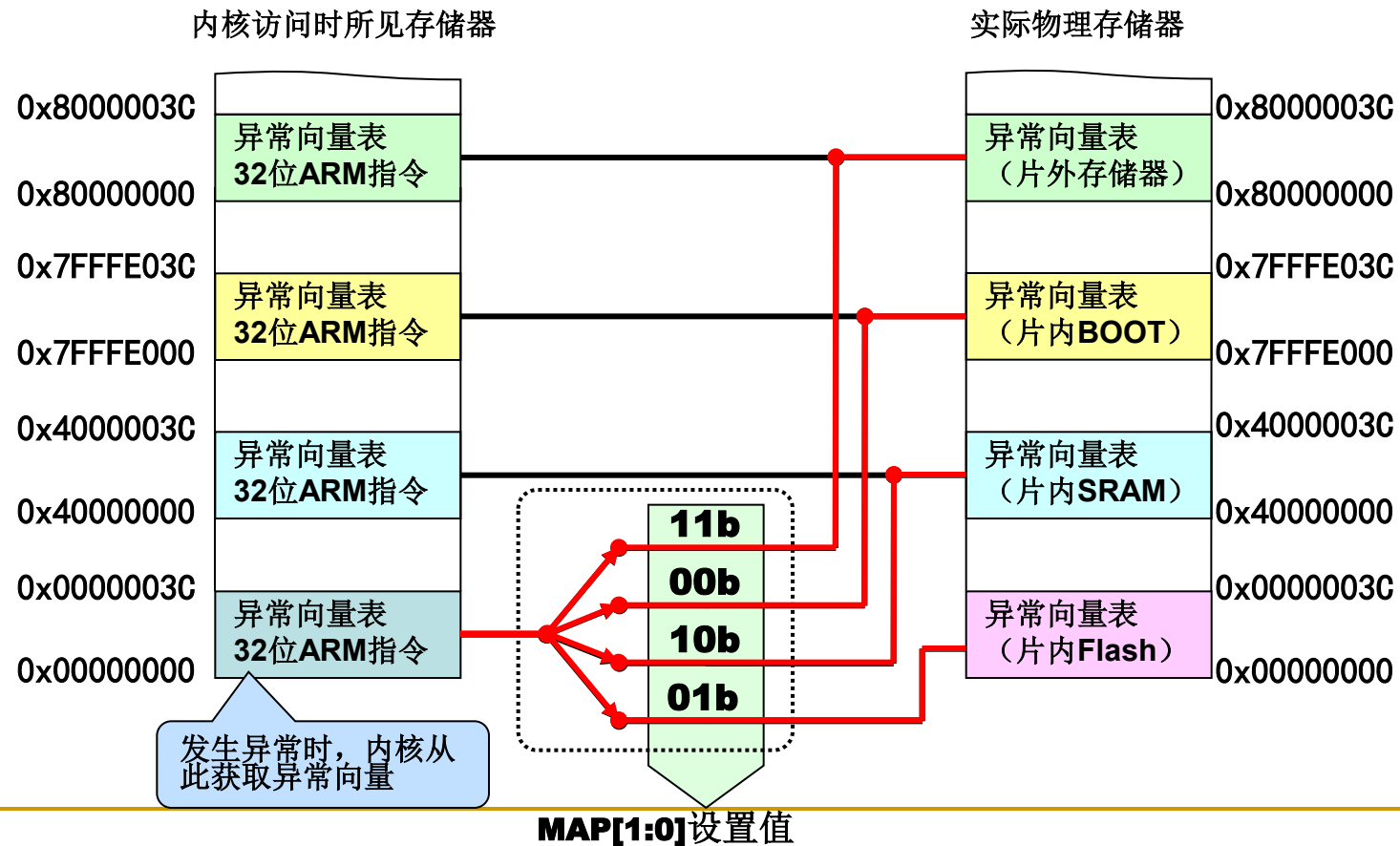
- 系统引导与存储器映射

对于LPC2200系列微处理器，当nRESET为低时，BOOT1:0脚的状态控制着引导方式。

P2.27/D27/ BOOT1	P2.26/D26/ BOOT0	引导方式	MAP1:0
0	0	CS0控制的8位存储器	11
0	1	CS0控制的16位存储器	11
1	0	CS0控制的32位存储器	11
1	1	内部Flash存储器	01

存储器映射控制的使用注意事项

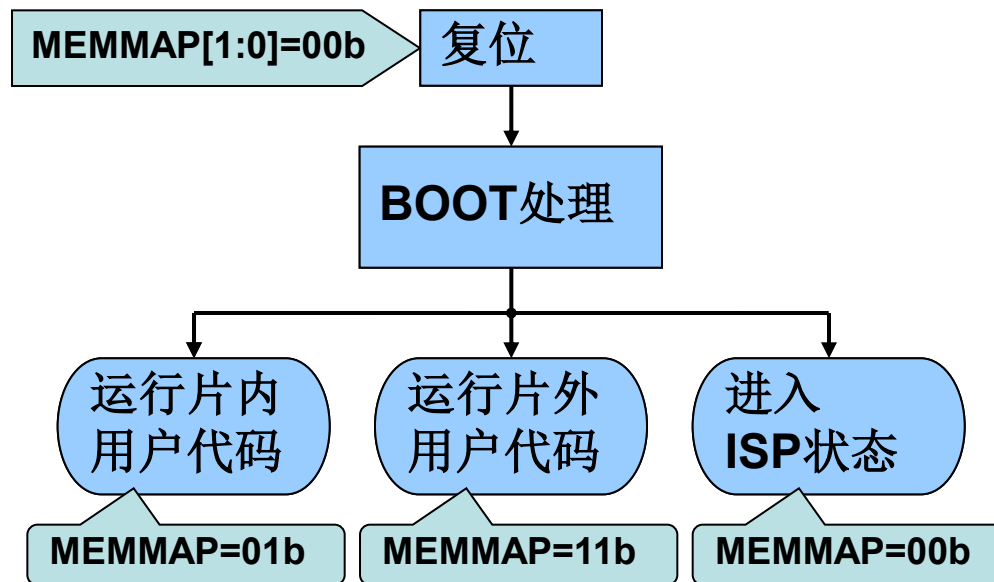
当从异常向量表地址访问到0x00000000地址。



4.4.8 存储器映射控制

• REMAP应用操作

通过MEMMAP寄存器，根据MEMMAP[1:0]的值，选择不同的启动模式。当MEMMAP[1:0]=00b时，系统进入ISP状态；当MEMMAP[1:0]=01b时，系统运行片内用户代码；当MEMMAP[1:0]=11b时，系统运行片外用户代码。通过MEMMAP寄存器，判断进入ISP还是启动用户程序。



4.4.9 功率控制

- 概述

节电模式	处理器	系统时钟	外设
空闲模式	停止执行指令	有效	正常工作
掉电模式	停止执行指令	无效	无需时钟支持的外设能够正常工作

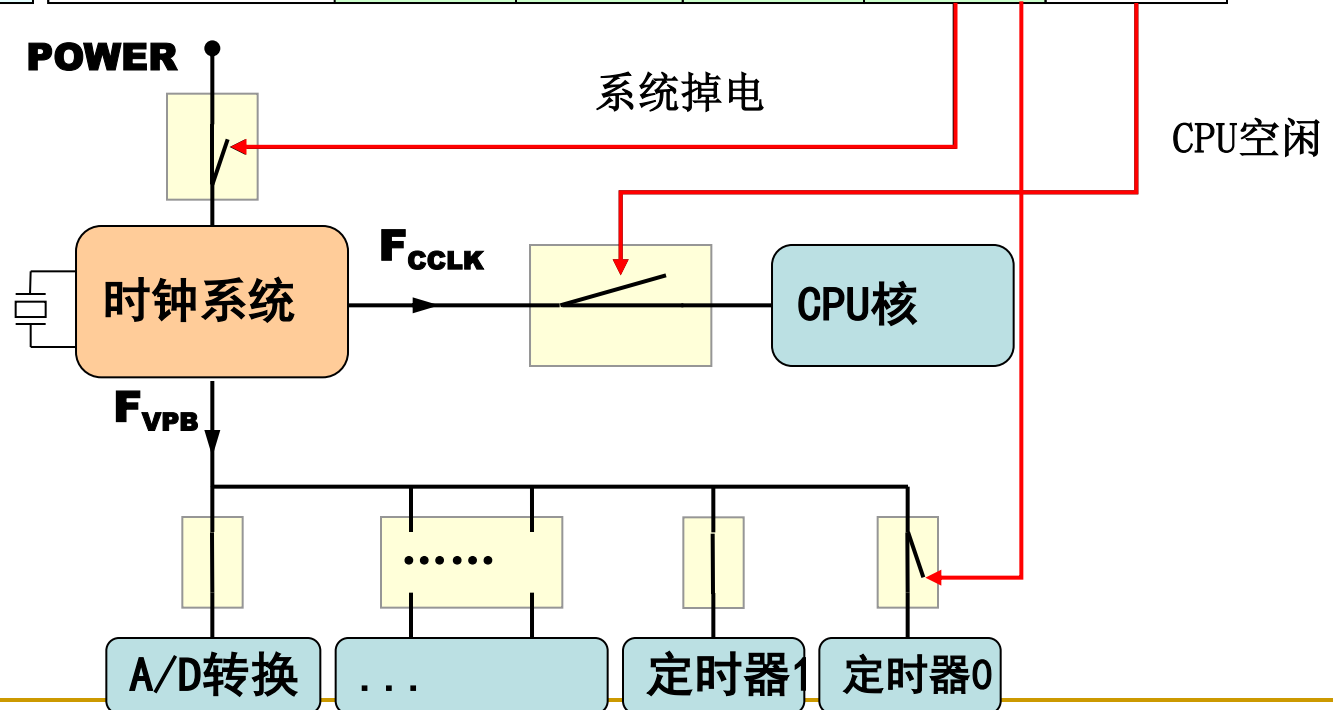
注：LPC2000系列芯片还允许程序对某个外设进行关闭控制。外设的功率控制特性允许独立关闭应用中不需要的外设，这样进一步降低了功耗。

寄存器描述

功率控制寄存器 (PCONP):

PCONP 中的每一位都对应一个外设 (通过门控使能时, 该位 GPIO 功能脚连接该外设和系统控制模块)。定时器 0 功能。

控制寄存器 PCONP	31 : 13	12	11 : 3	2	1	0
	—	PCAD		PCTIM1	PCTIM0	—



4.4.9 功率控制

- 功率控制注意事项

外部复位后，PCONP的值已经设置成使能所有接口和外围功能，所以用户不再需要去打开某个外设。

在需要控制功率的系统中，只要将应用中用到的外围功能的对应PCONP寄存器的位置1，寄存器的其它“保留”位或当前不需使用的外围功能对应寄存器中的位都必须清零。